

PLADICS: A Lightweight YOLO26n-Based Framework for River-Surface Plastic Waste Detection and Collection

by

Mst. Esat Jahan Akhi

Student ID: 2201022

Jannatul Ferdushi Ayesha

Student ID: 2201002

This capstone project report is presented to the
Department of Internet of Things and Robotics Engineering (IRE) at
University of Frontier Technology, Bangladesh,
in partial fulfillment of the requirements for the degree of
B.Sc. (Engg.) in Internet of Things and Robotics Engineering.

Supervised By

Suman Saha

Assistant Professor,

**Department of Internet of Things and Robotics Engineering
University of Frontier Technology, Bangladesh**



**Department of Internet of Things and Robotics Engineering
University of Frontier Technology, Bangladesh
Gazipur-1750, Bangladesh**

Date of Submission: 15th September, 2026

**Dedicated to our beloved
Mother and Father**

This capstone project report titled, “*PLADICS: A Lightweight YOLO26n-Based Framework for River-Surface Plastic Waste Detection and Collection*”, submitted by Mst. Esat Jahan Akhi (ID: 2201022) and Jannatul Ferdushi Ayesha (ID: 2201002), from the session 2022–2023, is hereby recognized and recommended by the under-signed for partial fulfillment of the requirements for the degree of B.Sc. (Engg.) in Internet of Things and Robotics Engineering at University of Frontier Technology, Bangladesh. The report completed by the aforementioned students has been assessed and approved as meeting the necessary criteria for the capstone book on this 15th day of September, 2026 .

Board of Examiners

Suman Saha
Assistant Professor,
Department of Internet of Things and Robotics Engineering
University of Frontier Technology, Bangladesh

Sadia Enam
Assistant Professor,
Department of Internet of Things and Robotics Engineering, University of Frontier
Technology, Bangladesh

Md.Rafiqul Islam
Lecturer,
Department of Internet of Things and Robotics Engineering, University of Frontier
Technology, Bangladesh

Saurav Chandra Das
Lecturer,
Department of Internet of Things and Robotics Engineering, University of Frontier
Technology, Bangladesh

Declaration

We hereby declare that this capstone project report entitled PLADICS: A Lightweight YOLO26n-Based Framework for River-Surface Plastic Waste Detection and Collection was exclusively prepared by us during our academic tenure at the Department of Internet of Things and Robotics Engineering, University of Frontier Technology, Bangladesh, as part of the requirements for the capstone project. The findings and conclusions presented in this report are the results of our independent efforts. Any materials, information, or references reproduced or obtained from external sources have been appropriately cited and acknowledged in accordance with academic standards. This capstone project report does not incorporate any previously published or authored work by any other party, nor has any portion of this content been submitted for academic credit towards any other degree, diploma, or academic qualification at any educational institution. Furthermore, we have duly acknowledged all primary sources of support and guidance received during the course of this research.

This research was conducted under the supervision and guidance of **Suman Saha**, Assistant Professor & Chairman, Department of Internet of Things and Robotics Engineering. We sincerely acknowledge his invaluable guidance, continuous support, and insightful feedback in shaping this work.

Mst. Esat Jahan Akhi

Student ID: 2201022

Jannatul Ferdushi Ayesha

Student ID: 2201002

Acknowledgment

First and foremost, all praise and gratitude are due to Almighty Allah, the Most Gracious, the Most Merciful, for granting us the strength, patience, and wisdom to successfully complete this capstone project. Without His divine blessings and guidance, this accomplishment would not have been possible.

We extend our deepest gratitude to our respected supervisor **Suman Saha**, Assistant Professor, Department of Internet of Things and Robotics Engineering, University of Frontier Technology, Bangladesh, for his unwavering support, continuous guidance, valuable suggestions, and insightful feedback throughout our research journey. His commitment to academic excellence, constructive criticism, and detailed attention have played an important part in the successful implementation of this capstone project throughout this work from dataset preparation and model evaluation to embedded deployment and prototype implementation.

We are very thankful to the Department of Internet of Things and Robotics Engineering, University of Frontier Technology, Bangladesh for providing the required academic environment, technical resources and support in this project. We also extend our thanks to our families, friends and fellow students (seniors and juniors) for their constant encouragement, patience and cooperation. Their inspiration and assistance were crucial to the successful completion of this capstone project.

Abstract

The increasing growth of aquatic plastic pollution is becoming a growing threat to freshwater ecosystems, highlighting the need for effective and reliable monitoring systems. However, many existing vision-based studies rely largely on dataset-specific evaluations and provide limited evidence regarding annotation quality, robustness, confidence reliability, and model interpretability. This capstone book presents PLADICS, an integrated framework for riverine plastic detection with collection-oriented system integration. A detailed dataset audit reduced 7,759 image-label pairs to 7,264 and removed 175 invalid bounding boxes. The cleaned single-class dataset was then used to compare YOLOv8n, YOLOv10n, YOLO11n, YOLO26n, YOLO26s, NanoDet-Plus-M, and RT-DETR-L, while separate multi-class experiments examined category granularity and transfer learning. The evaluation also included four-fold internal cross-validation, reduced-training-data analysis, confidence calibration, and detection-specific SHAP attribution. On the held-out test set, the 100-epoch YOLO26n checkpoint produced 0.919 Precision, 0.844 Recall, 0.915 mAP@0.5, and 0.608 mAP@0.5:0.95. Four-fold validation resulted in a mean mAP@0.5 of 0.860 ± 0.015 , while the reduced-data experiments showed a progressive decline in performance as the amount of training data decreased. The threshold-conditioned ECE was 0.1696, and the SHAP analysis showed strong deletion sensitivity but only partial spatial concentration within the detected regions. The selected detector was then connected to ByteTrack-assisted target continuity and rule-based collection logic. Overall, PLADICS provides a reproducible framework for evaluating and integrating river-surface plastic detection, deployed on a Raspberry Pi-based rover prototype, without assuming that the results will generalize universally to unseen waterways.

Acronyms

AI	Artificial Intelligence
AP	Average Precision
CI	Confidence Interval
CNN	Convolutional Neural Network
COCO	Common Objects in Context
CPU	Central Processing Unit
CSV	Comma-Separated Values
DETR	Detection Transformer
ECE	Expected Calibration Error
FPS	Frames Per Second
GFLOPs	Giga Floating-Point Operations
IoT	Internet of Things
IoU	Intersection over Union
JSON	JavaScript Object Notation
mAP	mean Average Precision
MQTT	Message Queuing Telemetry Transport
NCNN	Neural Network Inference Framework (Tencent)
PLADICS	Plastic Detection and Intelligent Collection System
MJPEG	Motion JPEG (Motion Joint Photographic Experts Group)
RiSID	River Surface Image Dataset for Instance Segmentation
RT-DETR	Real-Time Detection Transformer
SHAP	SHapley Additive exPlanations
SSD	Single Shot MultiBox Detector
UFTB	University of Frontier Technology, Bangladesh
XAI	Explainable Artificial Intelligence
YOLO	You Only Look Once
YOLOv8	You Only Look Once Version 8
YOLOv10	You Only Look Once Version 10
YOLO11	You Only Look Once Version 11
YOLO26	You Only Look Once Version 26

Contents

Declaration	iii
Acknowledgment	iv
Abstract	v
Acronyms	vi
List of Figures	ix
List of Tables	x
1 Introduction	1
1.1 Introduction	1
1.2 Research Background	2
1.3 Motivation	3
1.4 Objectives of This Work	4
1.5 Contribution of This Work	5
1.6 Organization of the Capstone Book	6
2 Literature Review	7
2.1 Deep Learning-Based Object Detection	7
2.2 Lightweight YOLO Model Approaches	8
2.2.1 YOLO-Based Methods for Floating Plastic Waste Detection	8
2.3 Transformer-Based Object Detection	10
2.4 Tracking-Assisted Detection Systems	10
2.5 Edge AI and Embedded Deployment	11
2.6 IoT-Based Environmental Monitoring	11
2.7 Existing Research Comparison	11
2.8 Research Gap Analysis	11
2.9 Summary	13
3 Methodology	14
3.1 Dataset Description	14
3.2 Dataset Cleaning and Annotation Refinement	16
3.3 Dataset Construction	17
3.4 Detection Model Selection	18
3.5 YOLO26 Architecture	18
3.6 Embedded Deployment	20
3.7 ByteTrack-Based Target Tracking	20
3.8 Rover Navigation Strategy	20
3.9 Plastic Collection Mechanism	25
3.10 IoT Monitoring and Data Logging	25
3.11 Hardware Implementation	27

3.12	Software Implementation	28
3.13	Validation, Robustness, and Explainability Procedures	29
3.14	System Workflow	30
3.15	Summary	30
4	Result Analysis and Discussion	31
4.1	Experimental Setup	31
4.2	Dataset Refinement Evaluation	32
4.3	Detection Model Comparison Across Architectures	33
4.4	Multiclass Riverine Plastic Detection Analysis	34
4.5	Transfer Learning Analysis	36
4.6	Cross-Validation Analysis	36
4.7	Robustness Under Reduced-Data Conditions	37
4.8	Final YOLO26n Model Performance	38
4.9	Confidence Calibration Analysis	40
4.10	Explainable AI Analysis	40
4.11	Qualitative Detection Analysis	42
4.12	Deployment-Oriented Evaluation	43
4.13	Prototype Integration and Proof-of-Concept Validation	43
4.14	Summary	45
5	Conclusion and Future Scope	46
5.1	Summary of the Work	46
5.2	Limitations and Future Scope of This Research	46
	Bibliography	48

List of Figures

3.1	Overall methodology framework of the proposed PLADICS system, showing dataset preparation, single-class and multiclass dataset construction, model training, architecture comparison, evaluation, deployment model selection, and rover-based system integration. . . .	15
3.2	Overview of the YOLO26 architecture consisting of backbone, neck, and detection head components used in this study.	19
3.3	YOLO26n–ByteTrack onboard detection pipeline.	21
3.4	Rule-based target selection, navigation, and collection control in the PLADICS rover.	22
3.5	Node-RED dashboard interface for real-time PLADICS monitoring.	26
4.1	Comparison of lightweight YOLO model performance after dataset refinement under the 50-epoch benchmark setting.	33
4.2	Configuration-level detection performance for the single-class and RiSID multiclass experiments.	35
4.3	Training convergence comparison between scratch training and pre-trained fine-tuning strategies.	36
4.4	Detection performance trend under reduced-data conditions.	38
4.5	Training and validation behavior of the refined YOLO26n model during 100-epoch optimization.	39
4.6	Confusion matrix of the 100-epoch YOLO26n model evaluated on the cleaned single-class test set.	39
4.7	Confidence and calibration analysis of the selected YOLO26n-100 model.	40
4.8	Representative detection-specific SHAP examples generated using the YOLO26n-100 checkpoint. Each example shows the letterboxed test image, fixed reference detection, raw signed attribution map, and smoothed attribution overlay. Positive target-confidence contributions are shown in red, while negative contributions are shown in blue.	41
4.9	Representative qualitative detection results obtained under real riverine environmental conditions.	42
4.10	Developed PLADICS prototype integrating onboard computing, perception, control, and plastic collection mechanisms.	44

List of Tables

2.1	Representative vision-based approaches for aquatic and riverine waste detection.	12
3.1	Dataset audit summary before and after refinement.	17
3.2	RiSID multiclass configurations used in the experiments.	18
3.3	Parameters used in the onboard PLADICS deployment.	25
3.4	Main parameters monitored and logged by PLADICS.	26
4.1	Default configuration used for the Ultralytics-based benchmark experiments.	32
4.2	Test-set performance before and after dataset refinement.	32
4.3	Twenty-epoch architecture-level comparison on the cleaned merged single-class dataset.	34
4.4	Class-wise multiclass detection performance using YOLO26n.	35
4.5	Scratch training versus pretrained fine-tuning performance.	36
4.6	Four-fold cross-validation performance of the YOLO26n configuration.	37
4.7	Reduced-data robustness of the 50-epoch YOLO26n configuration.	37
4.8	Performance of YOLO26n under 50- and 100-epoch training configurations.	38
4.9	SHAP faithfulness and spatial-alignment results of YOLO26n-100 ($n = 21$).	42
4.10	CPU workstation benchmark of the selected YOLO26 deployment configurations.	43

List of Algorithms

1	Candidate Acceptance and Primary Target Selection	23
2	Rover Navigation Decision	24
3	Collection Triggering Logic	24

Chapter 1

Introduction

1.1 Introduction

Plastic pollution in freshwater environments has become a significant environmental challenge due to its impact on aquatic ecosystems, biodiversity, water quality, and downstream marine environments [1, 2]. Among different sources of plastic pollution, river systems play an important role in transporting mismanaged plastic waste from urban and terrestrial regions toward lakes, estuaries, and oceans. Studies have shown that a limited number of rivers contribute significantly to ocean plastic transport, highlighting the importance of upstream monitoring and interception strategies [3]. Once present in aquatic environments, plastic debris can obstruct waterways, interact with aquatic organisms through ingestion or entanglement, and gradually fragment into smaller particles that persist within freshwater ecosystems [1]. The growth of plastic waste in river systems is ongoing and is increasing the demand for efficient monitoring and removal strategies. These should allow for reliable and sustainable operations.

Current methods of monitoring and collecting plastic waste in rivers are based on direct observation and manual retrieval, which are labor-intensive, time-consuming and costly to the environment [4]. These methods give direct information on the level of pollution, but using them extensively for continuous monitoring of large and complex aquatic environments is very difficult. Compared to controlled environments, river ecosystems are very dynamic and unpredictable. The visual aspects of plastic waste may all be significantly affected by changes in water flow, surface reflections, floating debris, seasonal effects, lighting, aquatic vegetation and environmental complexity. Therefore, an automated monitoring system should not only be able to detect plastics accurately but also be able to maintain consistent performance under different environmental conditions.

The detection of plastic garbage floating on river surfaces has become increasingly feasible because to recent developments in computer vision and deep learning. Even in complex environmental settings, deep learning-based item detectors can identify floating garbage by learning relevant visual patterns from annotated photos. For real-time visual detection, CNN-based detectors and "one-stage detection" techniques are popular because they produce good results with modest processing demands. Lightweight YOLO-based architectures in particular have attracted a lot of interest because they can strike a compromise between computational efficiency and detection accuracy [5, 6]. By adding lightweight backbones, attention mechanisms, sophisticated feature fusion algorithms, and architectural optimization strategies, several research have improved aquatic waste detection systems [7, 8]. The effectiveness of Transformer-based detectors (such RT-DETR), which can assess global contextual information from images, has also been investigated; how-

ever, their deployment on resource-constrained "edge devices" may be limited due to their high computing needs [9, 10]. Therefore, careful examination of detection skills, computational efficiency, tracking performance, and practical practicality is necessary for building an efficient AI-based waste management system.

PLADICS (Plastic Detection and Intelligent Collection System), a thorough framework for handling plastic debris floating on river surfaces, is presented in this capstone project. The system incorporates robotic collection mechanisms, embedded computing, IoT-based monitoring capabilities, and lightweight deep learning-based detection technology. It was created to get over the drawbacks of traditional image-based detection techniques, allowing the visual data that was found to immediately support garbage collecting activities. PLADICS combines artificial intelligence, embedded hardware, robotics, and communication technologies to transform computer vision research into a practical environmental monitoring solution. This architecture incorporates plastic identification, target tracking, decision-making, rover navigation, collection system control, and system monitoring into a smooth "identification-to-collection" workflow rather than handling identifying as a separate activity.

1.2 Research Background

Recent studies have demonstrated the potential of deep learning-based methods for environmental monitoring and floating waste detection. Convolutional Neural Networks (CNNs) have become widely used in visual recognition tasks because they can automatically extract hierarchical features from image data without relying on manually designed features. One-stage object detectors, including Single Shot MultiBox Detector (SSD), further enhanced real-time detection by combining object localization and classification within a single forward pass [5]. YOLO-based architectures have been extensively studied and evaluated among other deep learning-based detection techniques because of their capacity to successfully balance computational cost and detection performance. Significant advancements in backbone topologies, feature fusion strategies, attention processes, and optimization methodologies have resulted from ongoing work within the YOLO family. Research has been done on a number of lightweight YOLO-based techniques for aquatic waste identification with the goal of lowering processing requirements while maintaining reliable detection capabilities. [6, 7, 8]. Lightweight YOLO models are appropriate for embedded robotic systems with constrained processing power because of their effective inference capabilities. However, detection accuracy is not the only consideration when choosing a model for practical use; other considerations include model complexity, inference delay, memory consumption, and hardware limitations.

By using attention mechanisms to identify global links between distinct regions of a picture, transformer-based object detection techniques have offered another point of view for improving visual perception. By modeling these long-range dependencies, transformer-based methods can provide richer contextual information compared with approaches that mainly focus on local feature extraction. DETR-based architectures demonstrated the capability of transformer models to perform end-to-end object detection by reducing dependency on manually designed detection components. Real-time transformer-based detectors such as RT-DETR further im-

proved the practicality of transformer architectures by combining efficient feature extraction with transformer decoding mechanisms [9]. Recent studies have explored transformer-based approaches for aquatic and river-surface object detection, including LR-DETR and RTDETR-MARD, to improve contextual representation and multi-scale feature understanding [11, 10]. Although transformer-based models can provide strong detection capability, their higher computational requirements may create challenges for deployment on resource-constrained embedded platforms. Beyond detection accuracy, practical river plastic management systems require additional capabilities such as target tracking, decision-making, robotic control, and remote monitoring. Tracking-assisted detection approaches help maintain object identity across consecutive frames and provide temporal information for more reliable downstream actions. Previous studies have explored tracking-assisted aquatic waste detection by combining object detectors with tracking algorithms such as DeepSORT [12]. Similarly, IoT-based monitoring frameworks rely on lightweight communication protocols such as MQTT to support real-time telemetry collection, remote visualization, and system logging, particularly in resource-constrained environments [13, 14].

Even while the detection of floating plastics on river surfaces has advanced significantly, there are still a number of issues that need to be addressed. The evaluation of detection performance based on particular datasets and experimental frameworks has been the main focus of existing methods; dataset enhancement, system robustness across various training scenarios, confidence reliability, Explainable AI (XAI) analysis, and integration with useful waste collection methods have received relatively less attention. Additionally, there is little evidence of a connection between visual detection and real-world tasks like robotic navigation, autonomous decision-making, and garbage collection. These drawbacks highlight the necessity of an integrated framework that integrates IoT-based monitoring, robotic control, embedded system deployment, explainable evaluation, model comparison, and dependable data preparation. In light of this, the 'PLADICS' system was created with the goal of bridging the gap between deep learning-based perception (detection capabilities) and the actual plastic collection process by providing a thorough framework that includes the entire workflow, from detection to actual waste retrieval.

1.3 Motivation

The main motivation for this capstone project is the increasing need for automated and effective ways to handle floating plastic debris in waterways. Conventional garbage collection and monitoring techniques are frequently labor-intensive and time-consuming, which makes them inappropriate for ongoing, large-scale operations. Although the required data on pollution levels can be obtained by manual approaches, their long-term implementation in large and dynamic river settings poses serious difficulties. To ensure ongoing surveillance and improve the effectiveness of plastic waste management activities, an automated monitoring system is therefore crucial. Despite operating under restricted computational resources, such a system must be able to properly identify plastic garbage. The detection model must achieve the best possible balance between hardware needs, computational ef-

iciency, and accuracy for real-world deployment—especially when processing data or making "inference" on embedded devices with limited resources. This is crucial for robotic environmental monitoring systems since overall performance is directly impacted by processing power, processing speed, and response times. Therefore, to guaranty dependable and real-time functioning, choosing an appropriate and "lightweight" (low-resource) detection approach is essential.

Furthermore, the general problems of waste management cannot be solved by just recognizing waste. Although vision-based models are capable of identifying the existence and location of plastic garbage, their removal necessitates the use of additional capabilities, including ongoing tracking, decision-making, navigation control, waste collecting procedures, and thorough system monitoring. The results of detection cannot be immediately converted into practical environmental action without efficient coordination between these elements. In order to overcome this constraint, the project concentrates on developing an integrated framework as opposed to a standalone detection system, allowing the transformation of visual data into practical choices and real garbage collection activities. As a result, "PLADICS" has been created as a framework that is lightweight, simply repeatable, and easy to use; it goes beyond the constraints of image-based detection to provide a full workflow from trash collection to identification. This suggested system provides an efficient, automated way to manage floating plastic garbage on river surfaces by combining deep learning-based sensing, embedded computer, robotic control, and Internet of Things-based monitoring.

1.4 Objectives of This Work

The limitations of current techniques for identifying floating plastic on river surfaces have been taken into consideration when developing the goals of this capstone project. While improving detection accuracy has been the main focus of earlier solutions, practical implementation involves dealing with a number of other crucial factors, including dataset reliability, computational efficiency, model capability and robustness, embedded deployment, robotic decision-making, and system-level integration. Based on these specifications, the project seeks to provide a comprehensive framework that improves detection reliability, guaranties real-time operation, and enables useful applications in environmental monitoring, from detection to collection.

The following is a brief summary of this capstone project's main goals:

1. **Real-time Plastic Waste Detection and Classification:** In order to identify and track plastic waste in actual operating situations, a YOLO-based object identification model integrated with ByteTrack tracking on the Raspberry Pi platform was developed and put into use.
2. **Target Detection and Autonomous Navigation:** Using a zone-based navigation technique that incorporates control operations like turning left or right, moving forward, and pausing, the robotic system may move independently in the direction of recognized plastic objects. In order to minimize unnecessary movements brought on by detection noise or temporary target loss from the field of view, the system is built to track the target consistently over several frames.

3. **Automated Collection System:** When the rover reaches a stable position close to an identified plastic object, an automatic conveyor-belt-based collection system is activated. This system uses duplicate-prevention logic and cooling control to ensure that every identified plastic object is collected consistently and only once.
4. **Mission Data Storage and Real-time Monitoring:** installation of a remote monitoring system that uses MJPEG video streaming and MQTT-based telemetry to track system status in real-time, including temperature, FPS, CPU and RAM usage, detection data, and rover activities. For later performance evaluation and analysis, the system saves complete mission records, including CSV logs, screenshots, video footage, and mission status data.

1.5 Contribution of This Work

This capstone project investigates the full lifetime of an intelligent plastic monitoring system, in contrast to standard research that usually focuses on analyzing object detection performance. It includes a number of stages, from model evaluation and dataset preparation to embedded system deployment and physical prototype performance verification. The suggested system connects computer vision-based detection research with practical environmental applications by combining robotics, artificial intelligence, embedded computing, and Internet of Things-based monitoring into a single framework. This study takes into account the entire development and execution procedures necessary to set up an efficient plastic monitoring system rather than treating detection as a solo task.

The major contributions of this capstone project are summarized as follows:

- PLADICS is an integrated framework that combines robotic control, embedded inference, deep learning-based vision, physical collection, and Internet of Things monitoring into a single system architecture to identify and collect floating plastic on river surfaces.
- We developed single-class and multi-class dataset formats for annotation verification, class-mapping correction, invalid sample removal, and overall model evaluation, riverine plastic datasets have been rebuilt and refined.
- A comparative evaluation was conducted across lightweight YOLO-based detectors, compact CNN-based detection models, and transformer-based architectures. In addition, the proposed framework was assessed beyond conventional accuracy metrics through transfer-learning analysis, cross-validation, reduced-data robustness experiments, confidence calibration, and SHAP-based attribution analysis to examine model reliability and decision behavior.
- The selected YOLO26n model was deployed on a Raspberry Pi using NCNN, and ByteTrack-assisted target continuity, rule-based rover navigation, collection actuation, telemetry communication, and mission logging were integrated to demonstrate an end-to-end detection-to-collection prototype workflow.

1.6 Organization of the Capstone Book

The remainder of this capstone book is organized into five chapters. Each chapter presents a specific aspect of the proposed PLADICS framework, starting from the background and existing research, followed by the proposed methodology, experimental evaluation, and final conclusions.

Chapter 1 introduces the research background, motivation, objectives, and contributions of this capstone project. This chapter presents the importance of automated river-surface plastic detection, discusses the challenges associated with existing monitoring approaches, and provides an overview of the proposed PLADICS framework.

Chapter 2 presents the background study and literature review related to river-surface plastic detection, deep learning-based object detection, lightweight YOLO architectures, transformer-based detectors, tracking methods, embedded AI deployment, and IoT-enabled monitoring systems. This chapter discusses existing approaches, their limitations, and the research gaps addressed by this capstone work.

Chapter 3 describes the proposed PLADICS methodology in detail. This chapter covers dataset preparation, annotation refinement, dataset construction, detection model selection, YOLO26 architecture, embedded deployment, ByteTrack-assisted target tracking, rover navigation strategy, plastic collection mechanism, hardware implementation, software design, and IoT monitoring framework.

Chapter 4 presents the implementation details and experimental results of the proposed framework. This chapter discusses dataset refinement evaluation, architecture-level comparison, multiclass detection analysis, transfer learning evaluation, cross-validation, reduced-data robustness analysis, confidence calibration, SHAP-based explainability, deployment-oriented evaluation, and prototype validation.

Chapter 5 concludes the capstone work by summarizing the major findings, highlighting the achievements of the proposed PLADICS framework, discussing current limitations, and presenting possible future research directions for improving automated river-surface plastic monitoring and collection systems.

Chapter 2

Literature Review

Plastic pollution in river and freshwater environments has become an important research concern due to its impact on aquatic ecosystems, biodiversity, and water quality. Traditional monitoring and collection methods mainly depend on manual observation, which requires significant human effort and cannot provide continuous large-scale monitoring. In order to improve detection efficiency and enable long-term environmental monitoring, recent research has focused on creating automated monitoring techniques.

For the automatic detection of floating plastic on river surfaces, recent research has focused on computer vision and deep learning-based techniques. To improve automated monitoring capabilities in aquatic environments, studies have looked into transformer-based designs, embedded vision approaches, and lightweight object detection networks [6, 15]. Effective detection models, transformer-based methods, tracking-assisted tactics, embedded system implementation, and IoT-based monitoring frameworks are important study fields. Despite these developments, environmental variables and hardware limitations—such as surface light reflections, lighting fluctuations, object occlusion, complicated backgrounds, and the limited computational capability of edge devices—make real-world deployment difficult. These drawbacks suggest that choosing a high-performance detector alone is insufficient for an efficient system to identify plastic on river surfaces. Rather, a thorough pipeline that includes trustworthy dataset preparation, suitable model selection, implementation issues, decision-making techniques, and integration with actual garbage collection systems is needed. As a result, integrated intelligent systems that integrate perception, computing, and automated processes are replacing separate detection techniques in current research.

2.1 Deep Learning-Based Object Detection

Because deep learning allows models to automatically learn meaningful representations from image data, it has been crucial to the development of visual recognition systems. Because Convolutional Neural Networks (CNNs) can extract and encode complicated visual characteristics, they are commonly utilized for object recognition and categorization. CNN-based models learn hierarchical representations by recognizing low-level patterns, including edges and textures, and gradually building more abstract, object-level features, in contrast to traditional techniques that rely on manually designed features. Because of this capabilities, deep learning-based methods are ideal for the difficult conditions seen in rivers, where plastic objects might differ greatly in terms of size, shape, orientation, lighting, and background appearance. By combining localization and classification into a single forward pass, one-stage detectors like the Single Shot MultiBox Detector (SSD) have improved

real-time object detection and achieved a workable balance between accuracy and computational efficiency for environmental monitoring applications [5]. Through effective network design, feature fusion techniques, and optimal training procedures, the YOLO (You Only Look Once) family has further developed real-time detection systems in this context. YOLO-based models are frequently used in studies related to the identification of aquatic waste because of their quick inference times and comparatively inexpensive processing requirements. In contrast to "two-stage detectors," which require the creation of "area recommendations" (possible regions of interest) before classification, YOLO-based models finish the detection process in a single "forward pass," which lowers processing costs and allows real-time deployment.

Since detection models must function within the computational limitations of onboard hardware, inference speed and model efficiency are crucial factors for robots and embedded systems. As a result, while choosing a detection architecture, one should take into account aspects like computing cost, memory utilization, and deployment feasibility in addition to accuracy. In automated environmental monitoring systems, where AI-based perception must operate continually on devices with limited resources, these factors are especially important. Therefore, a useful detection model should balance the needs for real-world deployment with detecting capabilities.

2.2 Lightweight YOLO Model Approaches

For applications where models must function within constrained computational resources, lightweight object detectors are especially crucial. The overall performance of the system is not necessarily enhanced by increasing model complexity in real-world deployment settings, particularly on edge devices. Larger models may provide better detection accuracy, but they usually demand more power, memory, and processing capability, which presents extra difficulties for robotic and embedded platforms. Finding the ideal balance between detection accuracy, inference speed, computational efficiency, and application requirements is therefore the main objective in the development of lightweight detectors.

2.2.1 YOLO-Based Methods for Floating Plastic Waste Detection

Numerous YOLO-based object identification techniques have been studied for monitoring the aquatic environment and identifying floating waste. The following is a brief summary of the major developments and breakthroughs made in these techniques:

- **YOLOv5-Based Methods:** In order to reduce computing requirements without sacrificing detection efficiency, Chen and Zhu created an enhanced and lighter version of YOLOv5 by including ShuffleNetV2, GSConv, and attention methods. Using the Orca Flow-Img dataset, the model's mAP was 0.849. Nevertheless, surface reflections on the water and changes in lighting still had an impact on detection performance [7]. The results of this work show that for real-time applications in aquatic environments, a suitable

balance between computational efficiency and detection robustness must be struck.

- **YOLOv8-Based Methods:** Because of their effective designs and sophisticated feature extraction capabilities, YOLOv8-based techniques have attracted a lot of interest for the detection of floating waste. In order to identify different kinds of floating debris, Fan and his associates developed the SS-YOLOv8n algorithm, which achieved a mAP of 0.799 on the SLD dataset. Reliable detection is nevertheless hampered by elements including complicated backgrounds, partially submerged objects, and reflections from the water's surface. In order to improve multi-scale feature representation, Wang and his associates created the YOLOv8-MSS technique, which produced a mAP of 0.879 on the FloW-Img dataset. The impacts of sea waves and differences in item size continued to affect detection accuracy even though this strategy enhanced detection performance [16]. These findings show that environmental fluctuations and changes continue to be a significant obstacle to the practical application of technologies intended to identify garbage on river surfaces.
- **YOLOv7-Based Methods:** By adding more feature fusion and attention mechanisms, Peng et al. improved YOLOv7 for floating garbage identification. Although the suggested approach boosts the accuracy of object detection and localization, the computational complexity is increased by integrating these modules [17]. This study emphasizes the difficulty of striking a balance between improved detection capabilities and computing economy, which is especially important for embedded environmental monitoring systems.
- **YOLOv10 and YOLO11-Based Methods:** More recent iterations of YOLO, such as YOLOv10 and YOLO11, include structural improvements intended to boost computing effectiveness and improve small object detection. The continuous development of lightweight detection models appropriate for real-time applications is reflected in these developments. Because the particles to be detected are frequently small, partially visible, and sensitive to changing environmental circumstances, such improvements are essential for detecting floating plastic on river surfaces.
- **YOLO26-Based Detection:** YOLO26 is a cutting-edge, lightweight object detection architecture developed to provide efficient inference with minimal processing requirements. Its YOLO26n and YOLO26s variations achieve distinct accuracy-to-complexity ratios, allowing an evaluation of how model capabilities affect deployment efficacy under differing resource restrictions. YOLO26 was chosen as the main detector for this capstone project [18] because of its lightweight design and compatibility with embedded systems. Determining if a contemporary, lightweight detector can consistently complete detection tasks within the computing constraints of a robotic platform is made easier by evaluating various YOLO26 variations.

2.3 Transformer-Based Object Detection

By using attention mechanisms to understand global links among images, transformer-based object detectors have presented an alternate method to visual detection. An end-to-end object detection framework using a transformer-based "set prediction mechanism" was invented by DETR (Detection Transformer). Transformer designs may learn interactions between various portions of an image and capture long-range dependencies, in contrast to CNN-based approaches that mainly extract local information. In complicated scenarios, where detection efficiency depends on both the object's appearance and the surrounding visual information, this capacity improves contextual awareness. However, transformer models' sophisticated feature representation frequently requires a large amount of processing power, which may restrict their use on platforms with limited resources. By fusing effective feature extraction with transformer decoding, RT-DETR has modified transformer-based detection for real-time applications. These models provide strong detection capabilities, but high computational needs may limit their practical application in embedded systems, which have finite memory and processing power. Transformer-based techniques have been used in recent studies to detect aquatic garbage; for example, models such as LR-DETR and RTDETR-MARD have used multi-scale feature analysis and enhanced contextual representation to identify floating objects. However, when choosing models for real-time environmental monitoring activities, computational efficiency is still a crucial consideration [15, 11, 10].

2.4 Tracking-Assisted Detection Systems

Simply detection is not enough in video-based applications to deliver constant information about object movement. By preserving an object's identity across consecutive frames, tracking algorithms get around this restriction and eventually enable more trustworthy decision-making. For autonomous robotic systems, where individual detection events may be inconsistent because of things like transient occlusion, changes in lighting, water flow, or detector uncertainty, this feature is essential. Instead of responding to each detection event separately, the tracking system allows for more stable decision-making by preserving object identity across several frames. In the area of aquatic waste monitoring, research has been done on tracking-based techniques to ensure temporal consistency and support downstream applications. Nunkhaw and associates demonstrated the potential of combining detection and tracking for waste monitoring by combining YOLOv5 with DeepSORT to detect and track floating debris in a controlled laboratory setting [12]. In the same way, his coworkers assessed YOLOv8s' performance using photos of macro-plastics taken by UAVs and riverbanks, observing that occlusion continued to make it difficult to identify small and distant objects [19]. These investigations show that tracking offers useful temporal information, but it is still difficult to preserve trustworthy object identity in complicated environmental settings. ByteTrack and YOLO26n detection have been combined in this capstone project to preserve target continuity and support rover navigation and sample collection decision-making. The PLADICS system uses tracking data to provide more constant target knowledge over time, which improves the stability of following robotic actions rather of relying

solely on detections from individual frames [20].

2.5 Edge AI and Embedded Deployment

Deep learning models may now perform inference—the process of assessing results—directly on local devices thanks to edge AI, doing away with the requirement for distant computing servers. For automated environmental monitoring systems that operate in places where a dependable internet connection cannot always be ensured, this functionality is essential. For time-sensitive operations, local processing speeds up decision-making, improves system responsiveness, and lowers communication latency. However, because robotic systems usually have limited computing, memory, and power resources, implementing deep learning models on embedded platforms is quite difficult. Therefore, the dependable deployment of models on edge hardware depends on efficient model optimization and lightweight inference frameworks. By reducing memory utilization and improving speed on platforms like the Raspberry Pi, frameworks like NCNN offer optimal inference capabilities for devices with limited resources [18]. The chosen YOLO26n model is deployed on a Raspberry Pi 4 in PLADICS using NCNN for real-time detection; this allows the system to function in close proximity to the sensing device and makes it easier to coordinate AI-based perception and robotic control.

2.6 IoT-Based Environmental Monitoring

IoT technologies provide communication and monitoring capabilities for distributed environmental systems by enabling real-time data exchange, remote visualization, and operational tracking. Lightweight communication protocols such as MQTT are widely used in resource-constrained devices due to their low overhead and suitability for telemetry applications [14]. Node-RED-based dashboards further support real-time visualization of sensor data, mission status, and system information. In PLADICS, IoT monitoring is integrated to record detection information, rover status, and operational logs through a connected monitoring framework [21].

2.7 Existing Research Comparison

Table 2.1 presents a comparison of representative vision-based aquatic waste detection studies. The variations in detection models, datasets, proven performance, and limitations found in current techniques are briefly highlighted in this comparative comparison.

2.8 Research Gap Analysis

Even though earlier research has made advances in identifying floating plastics on river surfaces, a number of issues need to be resolved before these techniques can be successfully applied in extensive, practical environmental monitoring systems. Although current research has greatly increased detection accuracy, there hasn't been

Table 2.1: Representative vision-based approaches for aquatic and riverine waste detection.

Study	Model	Dataset	mAP	Key Limitation
Chen and Zhu (2024) [7]	Improved Lightweight YOLOv5	Orca Flow-Img Dataset	0.849	Limited robustness under strong illumination and reflective water conditions.
Fan et al. (2024) [8]	SS-YOLOv8n	SLD dataset	0.799	Performance remained affected by reflections, semi-floating objects, and complex scenes.
Wang et al. (2024) [16]	YOLOv8-MSS	FloW-Img	0.879	Water-wave interference and object-scale variation remained challenging.
Peng et al. (2024) [17]	Improved YOLOv7	River floating-waste dataset	0.863	Additional modules increased computational complexity.
Nunkhaw et al. (2024) [12]	YOLOv5 + DeepSORT	Laboratory flume dataset	0.880	Evaluation was limited to controlled laboratory conditions.
Tikász et al. (2025) [19]	YOLOv8s	River and UAV macroplastic dataset	0.770	Small and distant objects remained difficult under occlusion.
Sun et al. (2025) [10]	RTDETR-MARD	FloW and custom datasets	0.866	Higher computational demand limited embedded deployment suitability.
Badams et al. (2025) [22]	FloYO-Net	D_six dataset	0.828	Limited dataset diversity restricted generalization analysis.
Cao et al. (2026) [23]	SEDS-YOLOv8	Hybrid water-surface dataset	0.888	Generalization beyond augmented training conditions requires further evaluation.
Sumon et al. (2026) [24]	RT-DETR-X + Faster R-CNN Ensemble	FloW-Img	0.891	Ensemble inference increases computational requirements.
Xu et al. (2026) [25]	YOLOv12n	Nine-category riverine debris dataset	0.712	Multi-category recognition increases classification difficulty.
Jitnupong et al. (2026) [26]	YOLOv11	Floating-waste dataset	0.780	Small dataset size limited broader validation.
Zhou et al. (2026) [27]	ESAK-YOLO	Complex floating-waste dataset	0.889	Occlusion and background interference remained challenging.

Note: The reported values were reproduced from the cited studies and should not be considered as direct or controlled head-to-head comparisons, since the studies differ in terms of datasets, class definitions, data splits, hardware configurations, and evaluation protocols.

enough research done on the crucial practical aspects of their dependable deployment. More research is especially required in areas like validation of dataset quality, evaluation of model dependability and methodological robustness, and integration with physical systems. The following is a brief summary of the major research gaps found in the examined studies:

1. Most existing studies primarily emphasize detection accuracy, while limited attention has been given to analyzing dataset quality, annotation consistency, and the possible effects of labeling errors on model performance.
2. Robustness evaluation under different training conditions, reduced data availability, and variations in environmental scenarios remains limited.
3. Confidence calibration and model explainability are rarely considered in aquatic waste detection studies, despite their importance for understanding prediction reliability and decision-making behavior.

4. Many existing approaches primarily evaluate image-based detection performance, while the integration of tracking, navigation, physical collection, and monitoring components into a complete operational framework remains insufficiently explored.
5. Further deployment-oriented evaluation is required to examine the practical feasibility of real-time environmental monitoring applications on resource-constrained hardware.

2.9 Summary

This chapter examines earlier studies on the detection of plastics on river surfaces, including tracking-assisted detection techniques, transformer-based models, lightweight YOLO architectures, deep learning-based object recognition, Edge AI deployment, and IoT-based environmental monitoring. Through advancements in detection accuracy, computing efficiency, and real-time processing capabilities, the reviewed research show notable progress in the development of automated aquatic waste detection systems. However, there are still a number of issues with explainability, deployment efficiency, model robustness, dataset dependability, confidence assessment, and interaction with physical collection devices. While comprehensive detection-to-action frameworks—which connect visual perception with physical intervention—remain comparatively rare, the majority of current approaches mainly focused on the detection task itself. By combining dataset refining, comparative detector evaluation, explainable analysis, embedded deployment, robotic collection, and IoT-based monitoring into a single framework, PLADICS fills these gaps. As a result, the suggested method advances current research beyond simple image-based detection in the direction of a comprehensive, intelligent solution for the identification and gathering of plastics on river surfaces.

Chapter 3

Methodology

The PLADICS framework’s whole workflow, including dataset preparation, data refining, detection model selection, embedded deployment, target detection and tracking, rover navigation, plastic collection, and IoT-based monitoring, is described in this capstone project’s approach. This framework was created to integrate visual perception with physical activities and remote monitoring capabilities to provide an end-to-end process, from detection to collection. In order to improve reliability during model training and evaluation, procedures like annotation verification, class correction, and dataset restructuring are carried out after the collection and preparation of a dataset consisting of plastic waste floating on the river surface. Additionally, ”single-class” and ”multi-class” datasets were created to assess model performance under various classification settings and study diverse detection circumstances.

Following dataset preparation, a number of object detection architectures were assessed, including transformer-based detectors, compact CNN-based models, and lightweight YOLO-based detectors. The trained network was then converted into the NCNN format and deployed on an embedded platform based on a Raspberry Pi in order to prepare the chosen model for edge deployment. To ensure target continuity between successive frames, ByteTrack-based target tracking was linked with the deployed detection pipeline. The position, confidence score, and bounding-box data of the discovered object were used to produce rover movement orders using a rule-based navigation technique. Only after the target satisfied predetermined stability criteria was the data gathering process initiated. Furthermore, an IoT monitoring subsystem was incorporated to offer operational monitoring, mission logging, and real-time telemetry visualization.

Figure 3.1, which illustrates the relationships between dataset preparation, model evaluation, deployment, robotic control, and monitoring components, shows the workflow of the entire process.

3.1 Dataset Description

Two publicly accessible datasets, RiSID and the Riverine Plastic Litter Dataset, were used for the experiments in this work. These datasets were chosen because they are pertinent to the observation of floating plastic on river surfaces and because the annotations (named and classed) samples needed for ”supervised object detection” are readily available. The creation of a more reliable and broadly applicable detection model is facilitated by the use of various data sources, which bring greater variation in object appearance and environmental variables. Images of river surfaces often show visual fluctuations because of things like shifting sunlight, water velocity, object placement or arrangement, and backdrop intricacy. Therefore, com-

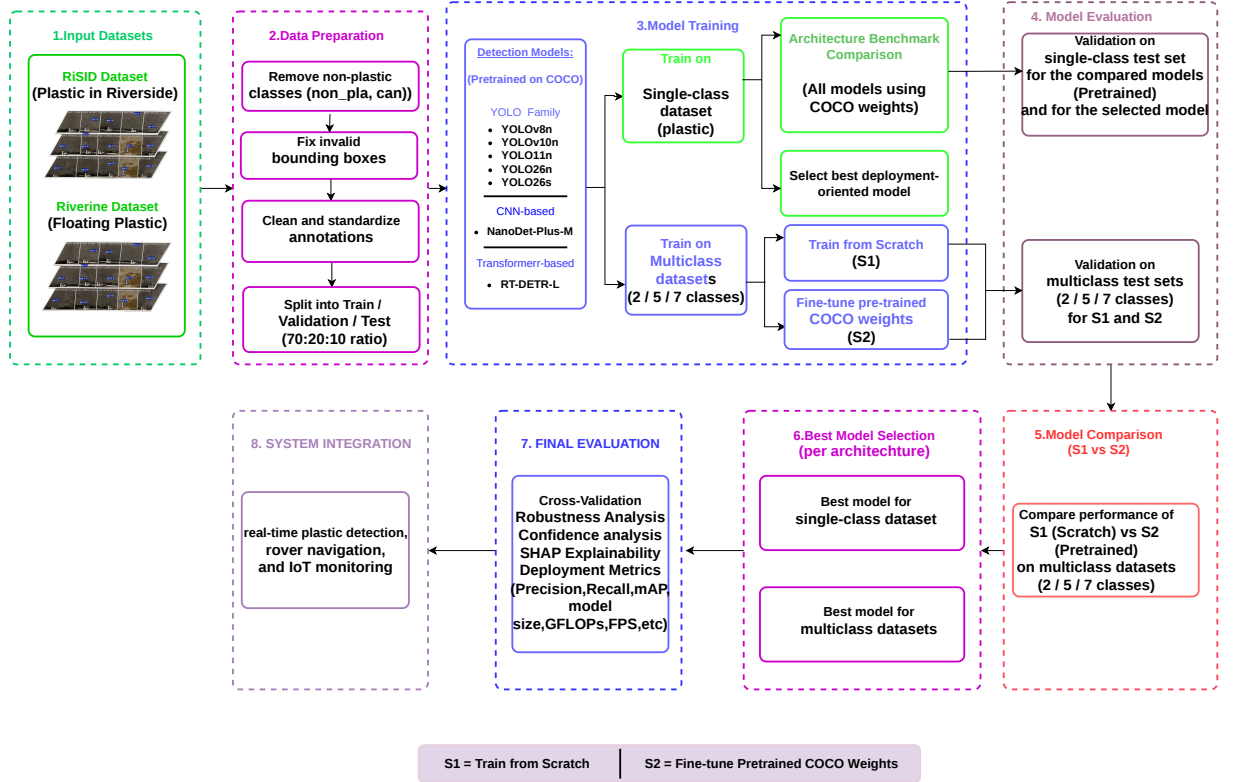


Figure 3.1: Overall methodology framework of the proposed PLADICS system, showing dataset preparation, single-class and multiclass dataset construction, model training, architecture comparison, evaluation, deployment model selection, and rover-based system integration.

binning multiple datasets creates a diverse training environment and supports both plastic presence detection and category-level recognition. Each dataset provides different advantages, where RiSID offers detailed river surface imagery with multiple annotation categories, while the Riverine Plastic Litter Dataset provides additional examples of common plastic objects. The main dataset sources are described below:

1. **RiSID Dataset:** RiSID (River Surface Image Dataset for Instance Segmentation) provides 7,356 images and 8,022 labeled debris instances collected from 11 locations across seven Japanese rivers, with COCO-format annotations available at multiple category granularities [28]. RiSID was selected as the primary dataset source because it provides relevant real-world river imagery and detailed instance-level annotations. The examination of object locations, size changes, and class distributions—factors essential for developing reliable object detection models—has been made possible by these annotations. Additionally, it is now feasible to assess model performance across a range of categorization difficulty levels due to the availability of different class configurations.
2. **Riverine Plastic Litter Dataset:** The Riverine Plastic Litter Dataset contains annotations for common plastic objects, including cans, plastic bags, bottles, boxes, and cups [29]. This dataset was incorporated to increase the number and diversity of plastic waste samples in the merged single-class dataset. The additional images introduced variations in object appearance, shape, and environmental conditions, supporting more robust single-class plastic detection.

3.2 Dataset Cleaning and Annotation Refinement

Before training, custom Python scripts were used to audit image–label correspondence, missing or empty labels, image readability, class IDs, annotation structure, and bounding-box validity. Detected annotation and class-mapping errors were corrected before dataset reconstruction [30]. The availability of annotated datasets such as FloW and RiSID has supported the development and evaluation of vision-based detection models for floating waste monitoring [31, 28]. Among the available Riverine annotations, 142 XML files had corresponding images in the `photo` and `test` directories. Only plastic categories were retained, while `can` was excluded. For RiSID, only `pla` annotations were retained because the initial single-class conversion had also mapped `non_pla` to plastic. The cleaned datasets were regenerated from the original COCO and XML annotations. Out-of-bound boxes were clipped when a valid positive-area region remained; otherwise, they were removed. Source-specific filename prefixes were used to prevent filename collisions during dataset merging.

Dataset refinement was considered an essential preprocessing step because annotation quality directly affects model training and evaluation. Incorrect class assignments, invalid bounding boxes, and inconsistent labels can introduce noise and reduce the reliability of performance measurements. Therefore, the refinement process was not limited to removing invalid samples; it also ensured consistency between image information and corresponding annotations. A mismatch between im-

Table 3.1: Dataset audit summary before and after refinement.

Metric	Before	After
Images	7759	7264
Labels	7759	7264
Missing labels	0	0
Empty labels	0	0
Invalid boxes	175	0
Corrupt images	0	0
Invalid class IDs	0	0

age content and labels may negatively affect the learning process because the model can receive incorrect supervision during training. Dataset auditing was performed before model training to reduce possible sources of uncertainty and to ensure that later performance differences were related to model behavior rather than annotation quality issues. The refined dataset was therefore used as the primary benchmark source for subsequent experiments. The resulting audit summary before and after refinement is reported in Table 3.1.

3.3 Dataset Construction

The constructed datasets were organized into single-class and multiclass configurations to evaluate both practical deployment requirements and classification complexity. The generated dataset formulations are described below:

1. **Single-Class Dataset:** For single-class detection, all retained RiSID and Riverine plastic annotations were mapped to class 0, **plastic**. This formulation was selected for rover deployment because the primary operational objective of the system is to identify the presence and location of collectible plastic objects rather than determine their specific categories. During real-time collection, the rover mainly requires target existence, horizontal position, and bounding-box area information to perform navigation and collection decisions. Although detailed plastic category recognition provides additional analytical value, it is not essential for immediate robotic action. Therefore, single-class detection was selected for deployment, while multiclass datasets were retained for experimental evaluation.
2. **Multiclass Dataset:** The RiSID two-, five-, and seven-class datasets were generated separately using the original category definitions [28]. These multiclass configurations were prepared to analyze how different levels of category complexity influence detection performance. While the single-class dataset supports practical rover operation, multiclass evaluation provides additional insight into the model’s ability to distinguish different plastic categories under varying classification conditions. The generated multiclass configurations are presented in Table 3.2.

Table 3.2: RiSID multiclass configurations used in the experiments.

Configuration	Categories
RiSID 2-class	pla, non_pla
RiSID 5-class	bottle, food_pack, bag, o_pla, non_pla
RiSID 7-class	bottle, o_bottle, food_pack, bag, o_bag, o_pla, non_pla

3.4 Detection Model Selection

The single-class benchmark included YOLOv8n, YOLOv10n, YOLO11n, YOLO26n, and YOLO26s. NanoDet-Plus-M and RT-DETR-L were added as compact CNN and transformer-based baselines to provide a broader comparison across different detection architectures. YOLO and RT-DETR experiments were conducted using the Ultralytics framework, while NanoDet-Plus-M was trained and evaluated using its original codebase [32, 33, 18, 34, 9]. The selected architectures represent different detection approaches, including lightweight one-stage detectors, compact CNN-based models, and transformer-based frameworks. Lightweight YOLO models were considered for their efficient real-time detection capability, NanoDet-Plus-M represented a compact CNN-based approach, and RT-DETR-L provided transformer-based detection with stronger contextual reasoning capability. It is now possible to choose a model by taking into account aspects of accuracy and practical application, such as model complexity, resource utilization, and implementation feasibility in embedded systems, thanks to this comparative analysis, which has improved understanding of the balance between detection capability and computational demands.

3.5 YOLO26 Architecture

Because of its lightweight design and adaptability for edge-based real-time applications, YOLO26 was chosen as the main detection architecture. YOLO26n was selected for embedded deployment because to its lower resource requirements. The YOLO26n and YOLO26s variants were assessed to examine the trade-off between detection performance and computing efficiency. The selected model was converted into NCNN format for optimized Raspberry Pi inference. YOLO26 follows a single-stage architecture consisting of backbone, neck, and detection head components, where feature extraction, multi-scale fusion, and object prediction are performed. The architecture incorporates C3k2 blocks, an enhanced SPPF module, and C2PSA attention for efficient feature representation. Additionally, its NMS-free inference strategy reduces post-processing overhead, making YOLO26 suitable for real-time plastic waste detection on resource- constrained robotic platforms [18]. The overall architecture is illustrated in Figure 3.2.

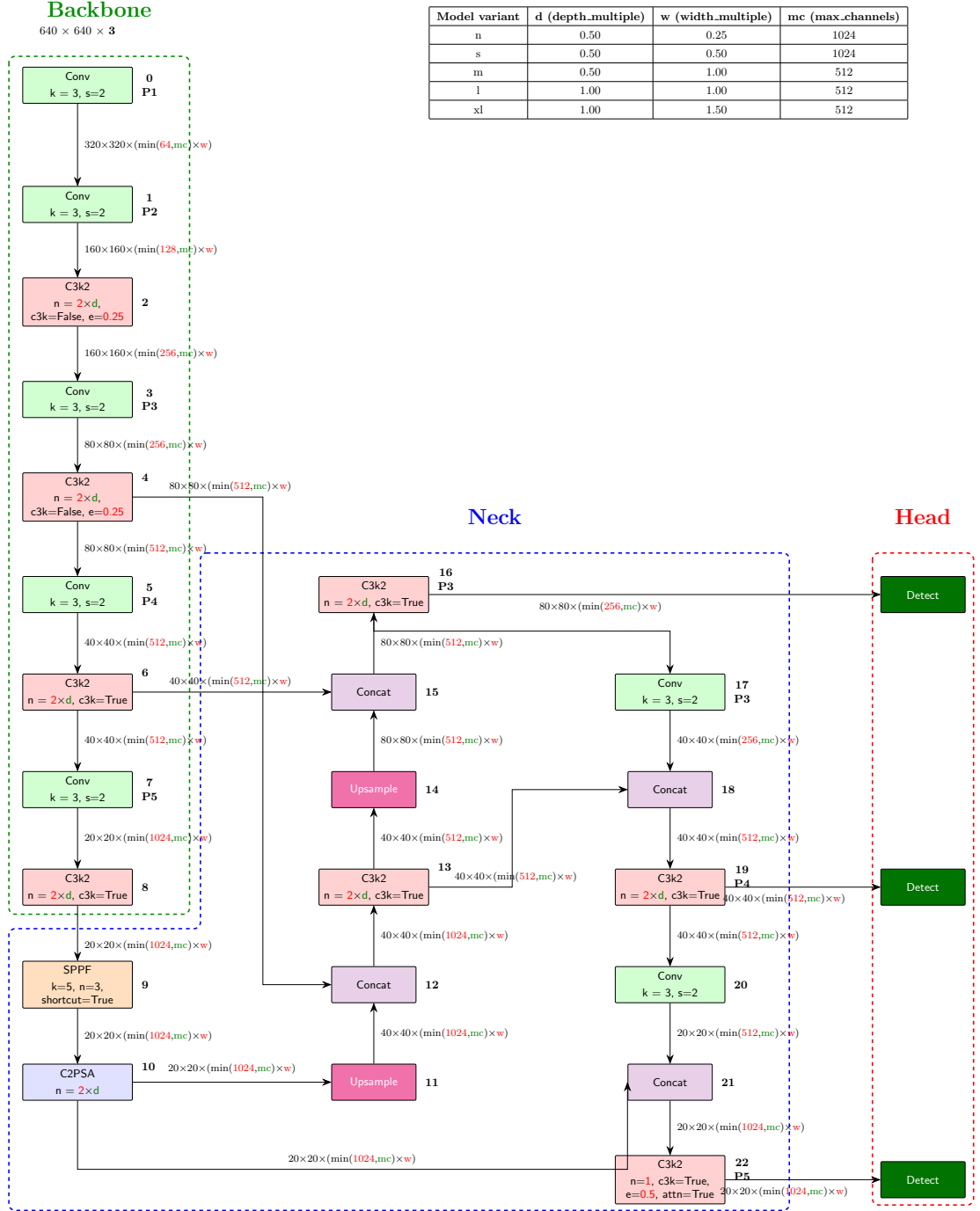


Figure 3.2: Overview of the YOLO26 architecture consisting of backbone, neck, and detection head components used in this study.

3.6 Embedded Deployment

The selected YOLO26n-100 model was exported to NCNN and deployed on a Raspberry Pi 4 with a Camera Module V2. Frames were captured at 640×480 pixels and processed at an inference size of 416 pixels. The deployment objective was to reduce dependency on external computing resources and enable local decision-making on the rover platform. Executing inference directly on the embedded device reduces communication delay and allows the system to operate in environments where continuous internet connectivity may not be available. Embedded deployment is a key stage of the PLADICS framework because it allows the final system to perform operations independently without depending on a remote computing server.

The Raspberry Pi-based deployment connects the trained deep learning model with the physical robotic platform by enabling direct communication between AI-based perception and rover control. By performing inference locally, the system reduces the delay between detection outputs and subsequent robotic actions, which supports real-time navigation and collection operations.

Tracking was introduced because individual frame-level detections alone are not sufficient to support reliable robotic actions. The tracking technology helps differentiate persistent targets from transitory detection noise and provides more consistent, dependable data for navigation and data-gathering decisions by preserving an object's unique identification over successive frames. In a river's dynamic environment, where objects may momentarily vanish from view due to water flow, partial obstruction, or detecting difficulties, this skill is essential.

3.7 ByteTrack-Based Target Tracking

ByteTrack with persistent tracking was used to associate detections across frames. Only detections belonging to the target class `plastic` were considered for navigation. A candidate i was accepted when

$$C_i \geq \tau_{\text{acc}}, \quad A_i \geq A_{\text{min}}, \quad \frac{A_i}{WH} \leq \rho_{\text{max}}, \quad (3.1)$$

where C_i denotes confidence and A_i denotes the candidate bounding-box area. The frame dimensions are represented by W and H ; τ_{acc} defines the minimum confidence threshold, A_{min} defines the minimum accepted box area, and ρ_{max} prevents excessively large bounding boxes from being selected. The tracking mechanism provides temporal information that is unavailable from individual detections, allowing the system to reduce unnecessary navigation changes caused by temporary detection fluctuations and improve stability during rover operation.

For each accepted candidate, the horizontal center position and bounding-box area were computed as

$$c_{x,i} = \frac{x_{1,i} + x_{2,i}}{2}, \quad A_i = (x_{2,i} - x_{1,i})(y_{2,i} - y_{1,i}). \quad (3.2)$$

3.8 Rover Navigation Strategy

When multiple candidates were available, the primary target was selected using

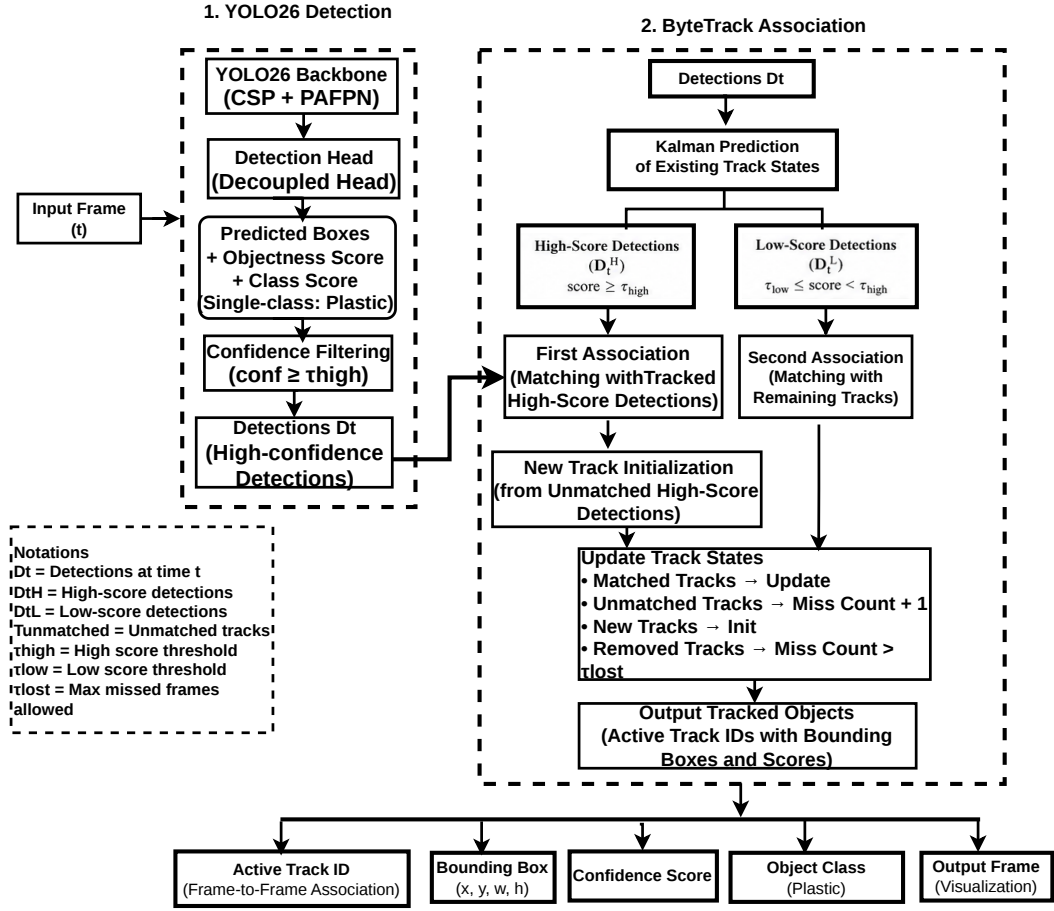


Figure 3.3: YOLO26n-ByteTrack onboard detection pipeline.

The navigation strategy was designed to convert visual detection outputs into meaningful rover actions. Instead of using a computationally expensive learning-based controller, a rule-based approach was selected because it provides transparency, lower computational complexity, and easier debugging during prototype development. The system relies on measurable visual features such as object position, bounding-box size, and temporal stability to generate rover commands.

$$S_i = w_C C_i + w_A \tilde{A}_i + w_G G_i + w_T T_i, \quad w_C + w_A + w_G + w_T = 1, \quad (3.3)$$

where

$$\tilde{A}_i = \frac{A_i}{\max_j A_j}, \quad G_i = \max \left(0, 1 - \frac{|c_{x,i} - W/2|}{W/2} \right). \quad (3.4)$$

The temporal-continuity term was defined as

$$T_i = \begin{cases} \eta_{id}, & \text{if the valid Track ID is unchanged,} \\ \eta_{prox}, & d_i \leq d_{\max} \wedge r_{\min} \leq A_i/A_{t-1} \leq r_{\max}, \\ 0, & \text{otherwise,} \end{cases} \quad (3.5)$$

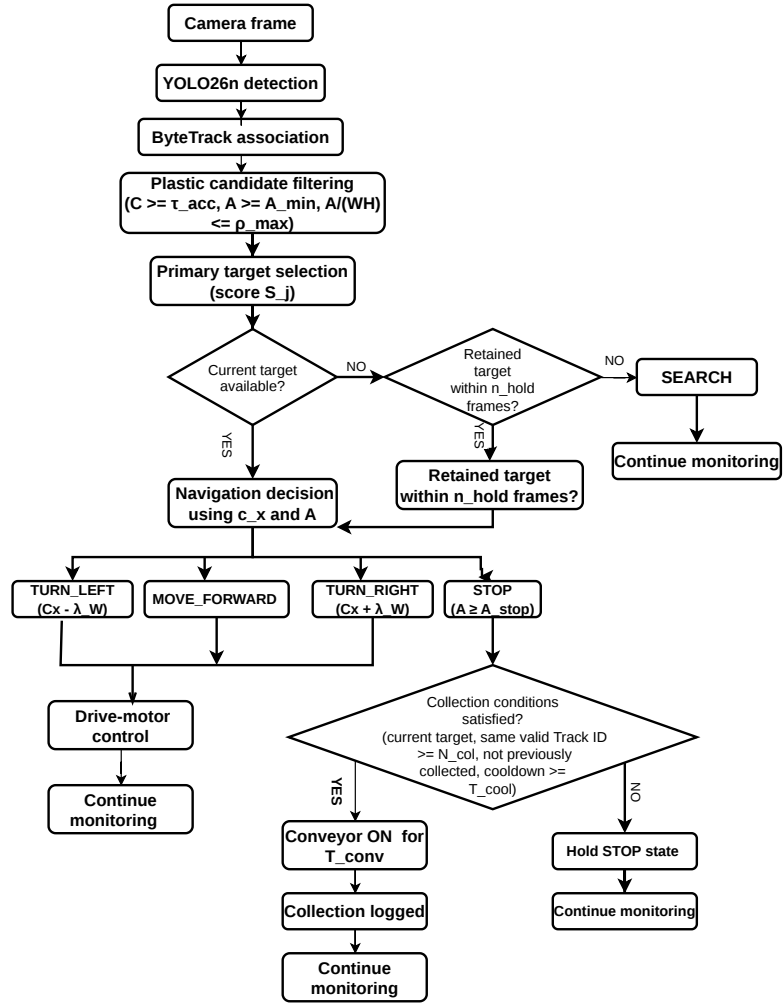


Figure 3.4: Rule-based target selection, navigation, and collection control in the PLADICS rover.

where d_i is the displacement from the previously selected target. The second condition provides spatial continuity when a valid Track ID is unavailable; it does not recover or reassign a ByteTrack identity. If no current candidate was available, the last valid target state was retained for at most N_{hold} processed frames. This retained state was used only for short-term navigation continuity and was not permitted to trigger collection.

The rover navigation command was determined by the horizontal target position and projected bounding-box area:

$$a_t = \begin{cases} \text{STOP}, & A_t \geq A_{\text{stop}}, \\ \text{TURN_LEFT}, & A_t < A_{\text{stop}} \wedge c_{x,t} < \lambda_L W, \\ \text{TURN_RIGHT}, & A_t < A_{\text{stop}} \wedge c_{x,t} > \lambda_R W, \\ \text{MOVE_FORWARD}, & \text{otherwise.} \end{cases} \quad (3.6)$$

If no target remained after the retention period, the rover entered the **SEARCH**

state. Movement commands were smoothed over the recent action history, while **STOP** was applied immediately. The L298N controller then translated the selected command into forward or turning motion; during **SEARCH**, both motors remained stopped.

Algorithm 1 summarizes the per-frame candidate acceptance and primary-target selection procedure described above, and Algorithm 2 summarizes the resulting navigation decision.

Algorithm 1 Candidate Acceptance and Primary Target Selection

Require: Detections $\{(C_i, x_{1,i}, y_{1,i}, x_{2,i}, y_{2,i}, \text{ID}_i)\}$ for class **plastic**; frame size $W \times H$

Ensure: Selected primary target (or None)

- 1: $\mathcal{A} \leftarrow \emptyset$
- 2: **for** each detection i **do**
- 3: $A_i \leftarrow (x_{2,i} - x_{1,i})(y_{2,i} - y_{1,i})$
- 4: **if** $C_i \geq \tau_{\text{acc}}$ **and** $A_i \geq A_{\min}$ **and** $A_i/(WH) \leq \rho_{\max}$ **then**
- 5: $\mathcal{A} \leftarrow \mathcal{A} \cup \{i\}$ ▷ Eq. (3.1)
- 6: **end if**
- 7: **end for**
- 8: **if** $\mathcal{A} = \emptyset$ **then**
- 9: **return** None
- 10: **end if**
- 11: **for** each $i \in \mathcal{A}$ **do**
- 12: Compute $c_{x,i}, \tilde{A}_i, G_i$ ▷ Eqs. (3.2), (3.4)
- 13: Compute continuity term T_i ▷ Eq. (3.5)
- 14: $S_i \leftarrow w_C C_i + w_A \tilde{A}_i + w_G G_i + w_T T_i$ ▷ Eq. (3.3)
- 15: **end for**
- 16: **return** $\arg \max_{i \in \mathcal{A}} S_i$

Algorithm 2 Rover Navigation Decision

Require: Selected target state $(c_{x,t}, A_t)$, or **None** if unavailable**Ensure:** Navigation action a_t

```

1: if target is None then
2:   if hold counter  $< N_{\text{hold}}$  then
3:     Retain last valid target state; increment hold counter
4:   else
5:      $a_t \leftarrow \text{SEARCH}$ 
6:   end if
7: else
8:   Reset hold counter
9:   if  $A_t \geq A_{\text{stop}}$  then
10:     $a_t \leftarrow \text{STOP}$ 
11:   else if  $c_{x,t} < \lambda_L W$  then
12:     $a_t \leftarrow \text{TURN\_LEFT}$ 
13:   else if  $c_{x,t} > \lambda_R W$  then
14:     $a_t \leftarrow \text{TURN\_RIGHT}$ 
15:   else
16:     $a_t \leftarrow \text{MOVE\_FORWARD}$ 
17:   end if
18: end if
19: Smooth  $a_t$  over the last  $N_{\text{smooth}}$  frames (except STOP, applied immediately)  $\triangleright$ 
    Eq. (3.6)
20: return  $a_t$ 

```

Algorithm 3 Collection Triggering Logic

Require: Rover state a_t , target Track ID stability counter n_{stable} , last collection time t_{last} , current time t **Ensure:** Conveyor activation decision

```

1: if  $a_t = \text{STOP}$  and  $n_{\text{stable}} \geq N_{\text{col}}$  and target not previously collected and  $(t - t_{\text{last}}) \geq T_{\text{cool}}$  then
2:   Activate conveyor for duration  $T_{\text{conv}}$ 
3:   Mark target as collected; set  $t_{\text{last}} \leftarrow t$ 
4: else
5:   Take no collection action
6: end if

```

3.9 Plastic Collection Mechanism

Collection was triggered only when the rover was in the **STOP** state and the current target retained the same valid Track ID for at least N_{col} consecutive frames, had not been previously collected, and satisfied the cooldown condition. Once these conditions were fulfilled, the conveyor mechanism was activated for T_{conv} . The collection triggering logic is summarized in Algorithm 3, and the deployment parameters are listed in Table 3.3.

The collection mechanism was intentionally separated from the detection process to prevent premature activation caused by temporary false detections. The physical collection process is initiated only when sufficient confidence is obtained from both the perception and tracking modules. The Track ID stability condition confirms that the detected object remains a consistent target before activation, while the cooldown condition prevents repeated collection attempts for the same target within a short time interval. This design improves operational reliability by reducing the possibility of activating the mechanism due to detection noise or unstable tracking results.

Table 3.3: Parameters used in the onboard PLADICS deployment.

Parameter	Description	Value
τ_{det}	Detector confidence threshold	0.20
τ_{IoU}	Detector IoU threshold	0.45
τ_{acc}	Candidate acceptance confidence	0.40
A_{min}	Minimum accepted bounding-box area	1200 px
ρ_{max}	Maximum box-to-frame area ratio	0.55
w_C, w_A, w_G, w_T	Target-selection weights	0.40, 0.30, 0.15, 0.15
η_{id}	Same-ID continuity score	1.00
η_{prox}	Spatial-continuity score	0.65
d_{max}	Maximum continuity displacement	100 px
$r_{\text{min}}, r_{\text{max}}$	Accepted area-ratio range	0.45, 2.20
N_{hold}	Target-retention window	6 frames
λ_L, λ_R	Navigation-zone boundaries	0.33, 0.66
A_{stop}	Stop-area threshold	45000 px
N_{smooth}	Action-smoothing window	5 frames
N_{col}	Collection stability requirement	4 frames
T_{cool}	Collection-event cooldown	5 s
T_{conv}	Conveyor activation duration	5 s

3.10 IoT Monitoring and Data Logging

PLADICS used a local Mosquitto broker and MQTT messaging [14] to transmit system telemetry to a Node-RED dashboard [21]. Each mission generated a timestamped CSV log, while the annotated video stream was provided through a Flask MJPEG endpoint. The monitoring subsystem records important operational



Figure 3.5: Node-RED dashboard interface for real-time PLADICS monitoring.

parameters including detection information, active Track ID, confidence score, navigation action, mission state, conveyor status, inference time, and frame rate. These parameters provide a complete overview of system behavior during rover operation and are summarized in Table 3.4.

The monitoring subsystem extends the capability of PLADICS beyond local autonomous operation by providing visibility into system status, detection events, and mission progress. This information supports system debugging, performance analysis, error identification, and future deployment of the system in field environments. The collected telemetry also provides valuable information for future optimization of the detection, navigation, and collection modules, while improving the ability to evaluate long-term operational performance.

Table 3.4: Main parameters monitored and logged by PLADICS.

Parameter	Description
Detection count	Number of accepted plastic detections
Track ID	Active ByteTrack identifier
Confidence score	Confidence of the selected target
Navigation action	Current rover command
Mission state	Current operational state
Conveyor state	ON/OFF state of the collection mechanism
Inference time	Per-frame processing time
Frame rate	Processed frames per second

The physical prototype integrates computational, sensing, actuation, and communication components required for autonomous operation. The hardware architecture was designed by considering the requirements of real-time perception, mobility, plastic collection, and communication within a compact rover platform. Each hardware component was selected based on its specific role in the complete PLADICS

workflow, where onboard processing, environment sensing, movement control, collection capability, and telemetry communication must operate together to achieve a complete detection-to-collection system.

3.11 Hardware Implementation

The PLADICS prototype was implemented using an embedded robotic platform that integrates sensing, computation, motion control, and waste collection capabilities into a single operational system. The hardware architecture of PLADICS was developed to support real-time plastic waste detection, autonomous navigation, and automated collection under practical operating conditions. Because of its computational power, energy efficiency, and adaptability for embedded AI applications, the Raspberry Pi 4 has been used as the central processing unit. An on-board processing unit, vision sensors, a motor control interface, and a conveyor-based collection system make up the entire prototype hardware arrangement, which allows AI-based decision-making modules to efficiently communicate with the actual robotic platform.

The following is a description of the hardware elements incorporated into the PLADICS prototype:

- **Raspberry Pi Platform:** To run the deployed AI model and control the linked system components, a Raspberry Pi 4 was used as the "on-board inference" platform. This platform managed a MQTT telemetry client, a rule-based navigation controller, ByteTrack association logic, and the NCNN-exported YOLO26n-100 model during prototype operation. The Raspberry Pi 4 was employed as the embedded computing unit during deployment, while a workstation with 16 GB of RAM was only utilized for offline benchmarking. This job split made it possible to analyze the model's performance on the workstation while keeping the embedded system's operating environment realistic.
- **Camera Module:** The main sensing tool utilized to take pictures of the river's surface was a Raspberry Pi Camera Module V2. Before being sent into the detection pipeline, the camera's 640×480 pixel resolution frames were shrunk to a "inference size" of 416 pixels. These visual inputs were used for target localization, object detection, and later navigation-related decision-making.
- **System of Motor Control:** The rover's mobility is managed by an L298N twin H-bridge motor driver, which uses differential-drive motor signals to translate control orders into actual motion. The navigation module sends the controller a variety of commands, including `MOVE_FORWARD`, `TURN_LEFT`, `TURN_RIGHT`, and `STOP`, which it converts into the proper motor actions to control the rover's chassis. This interface connects the physical motion control system to the AI-based decision-making module.
- **Collection Process:** To physically gather the plastic items found on the water's surface, a conveyor-based collection device was incorporated inside the rover. This procedure was triggered for a certain amount of time, T_{conv} , if the

stability and cooldown requirements outlined in Section 3.9 were satisfied for a target. This design minimizes instances of the mechanism activating needlessly owing to brief detection faults by ensuring that the collection operation only starts after obtaining sufficient dependability in the data collected from the detection and tracking modules.

3.12 Software Implementation

Perception, tracking, decision-making, control, and monitoring modules are all integrated into a smooth processing pipeline by PLADICS's software architecture. After processing camera information through a series of modules, the software workflow produces tracking data, navigation commands, object detection results, and finally the rover's real activities. From detection to data capture, each module completes a specific duty and interacts with other parts to finish the workflow. By enabling independent testing, analysis, and improvement of individual components, this modular structure improves system maintainability and facilitates future growth.

The following primary modules constitute the software implementation:

- **Detection Pipeline:** In order to prepare the frames for an inference resolution of 416 pixels, the detection pipeline loads the YOLO26n-100 weights exported via NCNN and performs a number of image pre-processing processes, such as resizing and letterboxing. The YOLO26n inference pipeline is then applied to the pre-processed frames, and the resulting detections are filtered according to the acceptance criteria given in the equation (3.1). The output of this stage provides object location, confidence information, and detection results required for subsequent tracking and decision-making modules.
- **Target Tracking Module:** The tracking module integrates ByteTrack to assign and maintain persistent track identifiers across consecutive frames. The generated tracking information provides temporal continuity and supplies the continuity term T_i used in the target-selection score of Equation (3.3). This enables the system to maintain stable target information during rover operation and reduces unnecessary navigation changes caused by temporary detection fluctuations.
- **Navigation Controller:** The navigation controller implements the rule-based decision logic described in Equation (3.6). The controller uses target position, bounding-box information, and tracking stability to generate rover movement commands. The generated commands include `MOVE_FORWARD`, `TURN_LEFT`, `TURN_RIGHT`, and `STOP`. Action smoothing over N_{smooth} frames is applied to reduce unnecessary command fluctuations, while the `SEARCH` state is activated when no valid target is available.
- **Collection Control Software:** The collection control module manages conveyor activation based on the navigation state and target stability conditions. The software triggers the collection mechanism only when the rover reaches the `STOP` state and the detected target satisfies the required Track ID stability and cooldown conditions. This prevents unnecessary activation caused by temporary detection errors or unstable tracking results.

- **IoT Monitoring Dashboard:** Using a local Mosquitto broker, the monitoring software uses MQTT connectivity to send system telemetry data to a Node-RED dashboard. Detection results, active Track IDs, confidence scores, navigation actions, mission status, conveyor status, inference time, and frame rate are among the operational characteristics that are shown in real-time on this dashboard. Additionally, a Flask endpoint provides an annotated MJPEG video stream that allows operators to track mission progress and keep an eye on the rover’s activities in real time.

3.13 Validation, Robustness, and Explainability Procedures

To evaluate the robustness, reliability, and interpretability of the chosen YOLO26n model, additional evaluation processes were carried out in addition to typical detection metrics. These assessments looked at whether the model performs consistently under different training conditions, whether its confidence output is trustworthy, and whether its predictions are grounded on significant visual characteristics. Cross-validation, robustness analysis through data reduction, confidence calibration, and SHAP-based interpretability assessment were all part of the evaluation procedure.

- **Cross-validation:** Four-fold cross-validation was performed using the 50-epoch YOLO26n configuration. The initial test set was not included in the cross-validation procedure, and each fold was evaluated against its assigned validation subset. This technique was used to confirm that the model’s performance was consistent across different internal data splits.
- **Reduced-data robustness:** The training set was reduced to 75%, 50%, and 25% of its original size, while the validation and test partitions remained unchanged. This analysis was performed to evaluate the sensitivity of model performance to reduced training data availability.
- **Confidence calibration:** Predictions from YOLO26n-100 were retained at a confidence threshold of 0.25 and matched one-to-one with ground-truth boxes at an IoU threshold of 0.50. Ten equal-width confidence bins were used to compute Expected Calibration Error (ECE) [35, 36]. This analysis evaluated whether the predicted confidence scores accurately reflected the actual prediction reliability.
- **SHAP analysis:** SHAP attribution was evaluated on 21 held-out test images using YOLO26n-100 [37]. For each image, the highest-confidence plastic detection above 0.25 was selected as the reference target. Perturbed predictions were matched to the same class at an IoU threshold of 0.30. Faithfulness was evaluated by masking the top 20%, 40%, and 60% of positive-attribution pixels, while spatial alignment was measured using attribution mass, top-5% overlap, pointing-game, and attribution-centroid criteria.

3.14 System Workflow

Every captured frame goes through the entire PLADICS processing pipeline at runtime; the YOLO26n model is used for image acquisition and detection, and then ByteTrack-based target association and selection logic is applied (as explained in sections 3.7–3.9). Target position, confidence score, bounding-box features, and temporal stability are among the parameters that are taken into consideration while translating the gathered visual information into navigation decisions. The motor controller is driven by the resulting navigation command, and once the required stability and cooldown conditions are satisfied, the collection mechanism operates on its own. In parallel, system telemetry parameters are published through MQTT for real-time visualization and recorded in timestamped CSV logs for post-mission analysis. This workflow establishes a complete closed-loop system where visual perception, decision-making, physical action, and monitoring operate together as an integrated detection-to-collection framework.

3.15 Summary

This chapter presented the complete methodology of the PLADICS framework, covering dataset preparation, annotation refinement, dataset construction, detection model selection, embedded deployment, target tracking, rover navigation, plastic collection, hardware implementation, software integration, and IoT-based monitoring. The proposed methodology establishes a complete connection between artificial intelligence-based perception and physical robotic action by integrating multiple components into a unified detection-to-collection pipeline. Unlike traditional detection-only approaches, PLADICS considers the complete lifecycle of an intelligent environmental monitoring system, including reliable data preparation, efficient model selection, deployment constraints, decision-making, physical interaction, and operational monitoring. This integrated approach provides a practical foundation for prototype-level autonomous river plastic monitoring and collection. The next chapter presents the experimental evaluation of the developed framework and discusses the performance of the proposed approach under different experimental settings.

Chapter 4

Result Analysis and Discussion

The experimental results of the proposed PLADICS framework are presented and analyzed in this chapter. Refinement of datasets, comparison of detection models, multi-class detection, transfer learning analysis, robustness verification, confidence calibration, Explainable AI (XAI) analysis, deployment efficiency, and prototype validation are all included in the evaluation’s scope. These tests were carried out to evaluate the chosen models’ detection accuracy as well as their dependability, computational effectiveness, decision-making procedures, and appropriateness for real-time embedded deployment. Standard object detection measures were used to compare different designs. Further experiments were conducted to examine model stability, prediction reliability, and practical operational efficacy.

4.1 Experimental Setup

YOLO and RT-DETR-L were executed with Ultralytics and PyTorch, whereas NanoDet-Plus-M retained its original evaluation pipeline. All benchmark runs were CPU-based on Windows 11 using an Intel[®] Core[™] i5-1235U system with 16 GB RAM. The software stack comprised Python 3.11.9, PyTorch 2.10.0+cpu, Torchvision 0.25.0, OpenCV 4.13.0, and Ultralytics 8.4.36. Except where an experiment required a different setting, training used 640×640 inputs, batch size 8, and 50 epochs, while YOLO26n was additionally extended to 100 epochs. Clean datasets followed a 70:20:10 train-validation-test division. Test subsets were used for the principal benchmark, multiclass, transfer-learning, robustness, calibration, and final-model evaluations, while cross-validation used only the held-out validation fold and did not reuse the original test set.

Detector quality was measured using Precision, Recall, mAP@0.5, and mAP@0.5:0.95. Parameter count, GFLOPs, checkpoint size, inference latency, and FPS were recorded as efficiency indicators. These timing measurements refer to the CPU workstation rather than the Raspberry Pi deployment platform. The purpose of this experimental setup was to maintain uniformity in dataset splitting, input resolution, and evaluation metrics across various evaluation scenarios. This controlled setup enables a fair comparison among different architectures and facilitates analysis of the separate effects of model selection, training duration, and dataset size. Table 4.1 lists the main benchmark settings.

Table 4.1: Default configuration used for the Ultralytics-based benchmark experiments.

Parameter	Value
Framework	Ultralytics 8.4.36 (YOLO and RT-DETR-L)
Python Version	3.11.9
Operating System	Windows 11 (64-bit)
Deep Learning Library	PyTorch 2.10.0+cpu
Annotation Format	YOLO detection format
Input Image Size	640×640
Batch Size	8
Training Epochs	Experiment-dependent: 20, 50, or 100
Dataset Split	70% training, 20% validation, 10% test
Device	CPU

4.2 Dataset Refinement Evaluation

Dataset refinement was evaluated by training the same detector set separately on the original merged dataset and the reconstructed clean dataset. The performance of each model was then measured on the corresponding held-out test split to analyze the effect of annotation correction and dataset reconstruction on detection capability.

Table 4.2: Test-set performance before and after dataset refinement.

Dataset State	Metric	YOLO10n	YOLO8n	YOLO11n	YOLO26n	YOLO26s
Before Cleaning	Precision	0.635	0.734	0.851	0.836	0.844
	Recall	0.688	0.692	0.745	0.738	0.722
	mAP@0.5	0.671	0.728	0.807	0.793	0.786
	mAP@0.5:0.95	0.339	0.375	0.424	0.461	0.474
After Cleaning	Precision	0.623	0.858	0.668	0.859	0.859
	Recall	0.718	0.797	0.746	0.797	0.813
	mAP@0.5	0.693	0.868	0.767	0.866	0.876
	mAP@0.5:0.95	0.431	0.562	0.482	0.556	0.580

Note: All reported values are test-set results. The before-cleaning results were obtained from the unrefined dataset test split, whereas the after-cleaning results were obtained from the reconstructed clean dataset test split. Since dataset refinement removed contaminated and invalid samples, the two test subsets differ in size and composition.

The cleaned-data runs in Table 4.2 generally produced higher detection scores. YOLOv8n obtained 0.868 mAP@0.5 and 0.562 mAP@0.5:0.95, YOLO26n reached 0.866 and 0.556, and YOLO26s recorded the highest 50-epoch cleaned-data values at 0.876 and 0.580, respectively. However, because the refinement process changed the number and composition of test samples, these differences represent the combined effect of annotation correction, class filtering, and dataset reconstruction rather than the isolated effect of a single cleaning operation.

The improvement after dataset refinement indicates that annotation quality has a direct influence on model learning and evaluation reliability. Incorrect labels, invalid bounding boxes, and inconsistent class assignments can introduce noise during training and may cause the model to learn incorrect visual patterns. Therefore, dataset auditing was treated as a fundamental component of the PLADICS framework rather than a simple preprocessing operation. The results demonstrate that model performance depends not only on architecture selection but also on the quality and consistency of the training data.

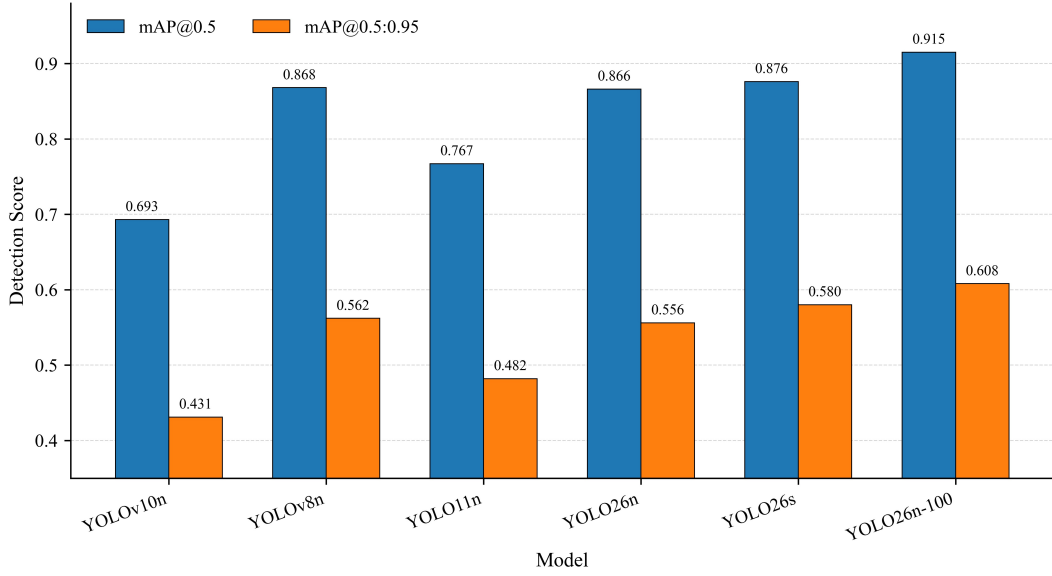


Figure 4.1: Comparison of lightweight YOLO model performance after dataset refinement under the 50-epoch benchmark setting.

Fig. 4.1 summarizes the lightweight YOLO model performance under the 50-epoch refined-dataset benchmark. YOLOv8n, YOLO26n, and YOLO26s achieved the strongest performance among the evaluated lightweight models.

4.3 Detection Model Comparison Across Architectures

The cleaned single-class dataset was also used for a cross-family benchmark covering YOLO detectors, the compact CNN-based NanoDet-Plus-M, and the transformer-based RT-DETR-L. Both detection quality and computational cost were considered through accuracy metrics, parameters, GFLOPs, checkpoint size, latency, and FPS.

A 20-epoch training budget was used for this comparison. Because RT-DETR-L and NanoDet-Plus-M followed their framework-specific implementations, the experiment should be viewed as a practical cross-architecture benchmark rather than a fully controlled implementation-level ablation.

Table 4.3 presents the 20-epoch architecture-level results. RT-DETR-L achieved the highest detection performance among the models with complete Precision and Recall reporting, reaching 0.858 mAP@0.5 and 0.554 mAP@0.5:0.95, with a Precision of 0.861 and Recall of 0.808. Within the YOLO family, YOLO26s achieved the

Table 4.3: Twenty-epoch architecture-level comparison on the cleaned merged single-class dataset.

Model	Architecture	Params	GFLOPs	P	R	mAP50	mAP50-95	FPS	Infer. (ms)	Size (MB)
RT-DETR-L	HGNetv2-L + Transformer Decoder	31.99M	103.4	0.861	0.808	0.858	0.554	0.98	1021.41	63.14
NanoDet-Plus-M	ShuffleNetV2 + GhostPAN	1.17M	3.69	–	–	0.615	0.354	1.51	664.00	16.27
YOLO11n	YOLO	2.58M	6.30	0.563	0.628	0.615	0.346	6.81	146.77	5.21
YOLOv8n	YOLO	3.01M	8.10	0.618	0.661	0.648	0.371	6.33	158.05	5.95
YOLO26n	YOLO	2.38M	5.20	0.616	0.637	0.650	0.382	5.26	190.20	5.12
YOLO26s	YOLO	9.47M	20.50	0.642	0.618	0.655	0.384	2.20	454.23	19.36

Note: Inference time is reported in milliseconds per image, and model size is reported in MB.

NanoDet-Plus-M reports COCO-style AP metrics, while Precision and Recall were not directly provided by its native evaluation pipeline. The reported model sizes correspond to native framework checkpoint files and may include different serialization overheads.

highest mAP@0.5 value of 0.655, while YOLO26n achieved a comparable performance with lower computational requirements. YOLO26n reached 0.650 mAP@0.5 and 0.382 mAP@0.5:0.95 while using fewer parameters and lower computational cost compared with YOLO26s. NanoDet-Plus-M provided the lowest parameter count and GFLOPs, but its reported AP values remained below the YOLO26 variants. The results demonstrate that higher detection accuracy was generally associated with increased computational demand across the tested architectures.

Considering detection performance, model complexity, checkpoint size, and CPU inference efficiency, YOLO26n provided the most suitable balance for the resource-constrained objectives of PLADICS. Although RT-DETR-L achieved stronger detection accuracy, its higher parameter count, GFLOPs, and inference latency make it less suitable for embedded deployment. In contrast, YOLO26n maintained competitive detection performance with a significantly smaller model size and lower computational requirements. Therefore, YOLO26n was selected as the deployment candidate because the target application requires not only accurate detection but also efficient real-time inference on a compact robotic platform.

4.4 Multiclass Riverine Plastic Detection Analysis

Although the main deployment model follows a single-class plastic detection strategy, multiclass experiments were also conducted to study category-level behavior. For this purpose, YOLO26n was evaluated on cleaned RiSID 2-class, 5-class, and 7-class datasets. The objective was to examine class-level performance under different category configurations.

Table 4.4 shows that class-wise performance varied across the different category definitions. In the RiSID 7-class setting, Bottle achieved 0.881 mAP@0.5 and Bag achieved 0.860 mAP@0.5, whereas O_bottle and O_bag produced lower scores of 0.518 and 0.660, respectively. These variations indicate that some plastic categories remain more challenging due to differences in object appearance, visibility, and environmental conditions. Fig. 4.2 further shows that overall mAP@0.5 decreased from 0.815 in the 2-class configuration to 0.781 and 0.755 in the 5-class and 7-class configurations, respectively. In contrast, mAP@0.5:0.95 remained relatively comparable across the configurations, indicating that category granularity affects

Table 4.4: Class-wise multiclass detection performance using YOLO26n.

Dataset	Class	P	R	mAP50	mAP50-95
RiSID-2	Plastic	0.820	0.860	0.893	0.587
	Non-plastic	0.783	0.654	0.736	0.457
	Overall	0.801	0.757	0.815	0.522
RiSID-5	Bottle	0.724	0.727	0.781	0.527
	Food_pack	0.684	0.606	0.753	0.583
	Bag	0.599	0.636	0.652	0.420
	O_pla	0.780	0.804	0.831	0.539
	Non_pla	0.776	0.720	0.774	0.471
	Overall	0.724	0.727	0.781	0.527
RiSID-7	Bottle	0.788	0.819	0.881	0.645
	O_bottle	0.663	0.435	0.518	0.375
	Food_pack	0.708	0.667	0.758	0.609
	Bag	0.683	0.851	0.860	0.636
	O_bag	0.628	0.604	0.660	0.457
	O_pla	0.768	0.788	0.840	0.532
	Non_pla	0.817	0.717	0.768	0.521
	Overall	0.722	0.697	0.755	0.539

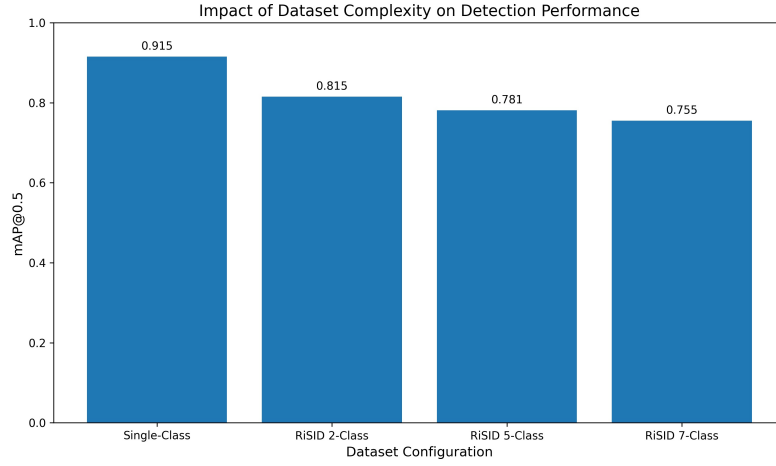


Figure 4.2: Configuration-level detection performance for the single-class and RiSID multiclass experiments.

different evaluation criteria in different ways.

The multiclass results demonstrate that increasing category complexity introduces additional classification challenges. Similar-looking objects, partial visibility, and environmental variations make fine-grained recognition more difficult compared with single-class detection. Although multiclass recognition provides additional information about plastic categories, the primary objective of the rover system is to identify and collect plastic objects rather than determine their exact category. Therefore, the deployment system adopts a single-class plastic detection strategy to reduce classification complexity and support faster decision-making during rover operation.

4.5 Transfer Learning Analysis

The transfer-learning experiment compared YOLO26n trained from scratch (S1) with the same model initialized from pretrained COCO weights (S2) on the RiSID 5-class configuration.

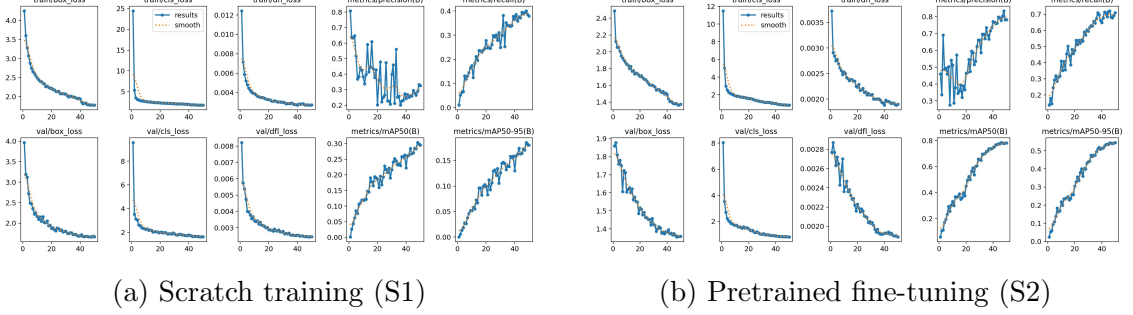


Figure 4.3: Training convergence comparison between scratch training and pre-trained fine-tuning strategies.

Table 4.5: Scratch training versus pretrained fine-tuning performance.

Training Strategy	P	R	mAP50	mAP50-95
S1 (Scratch Training)	0.290	0.401	0.303	0.185
S2 (Pretrained Fine-Tuning)	0.724	0.727	0.781	0.527

Pretrained initialization produced a large gain over scratch training: mAP@0.5 rose from 0.303 to 0.781, while mAP@0.5:0.95 increased from 0.185 to 0.527 (Table 4.5). The training curves in Fig. 4.3 also show slower convergence and weaker validation performance for scratch training, while the pretrained model reached higher and more stable validation metrics. The results demonstrate the importance of pretrained feature representations for environmental object detection tasks. Training from scratch requires learning visual features from limited domain-specific data, whereas pretrained initialization provides a stronger starting point by utilizing previously learned representations. This improvement is particularly important for environmental datasets, where collecting large amounts of domain-specific training data can be difficult. Transfer learning reduces the dependency on extremely large datasets by adapting previously learned visual features to the target environment.

4.6 Cross-Validation Analysis

The stability of the 50-epoch YOLO26n setting was further checked across four independently held-out validation folds.

Across the four folds, the mean Precision, Recall, mAP@0.5, and mAP@0.5:0.95 were 0.8311, 0.8028, 0.8596, and 0.5522, respectively. Fold-level mAP@0.5 varied between 0.8345 and 0.8750 with a relatively small standard deviation, indicating that the selected YOLO26n configuration maintained consistent performance across

Table 4.6: Four-fold cross-validation performance of the YOLO26n configuration.

Fold	P	R	mAP@0.5	mAP@0.5:0.95
Fold-1	0.8660	0.8200	0.8750	0.5711
Fold-2	0.8300	0.7907	0.8631	0.5497
Fold-3	0.7913	0.7746	0.8345	0.5426
Fold-4	0.8371	0.8257	0.8658	0.5456
Mean $\pm$ Std	0.8311 ± 0.0267	0.8028 ± 0.0210	0.8596 ± 0.0152	0.5522 ± 0.0112

Note: All folds were trained for 50 epochs. Metrics were calculated on the held-out validation partition of each fold. The reported values represent the mean and population standard deviation across four completed folds, while the original test subset was excluded from cross-validation.

different validation partitions (Table 4.6). These results demonstrate internal stability of the model under the evaluated dataset distribution; however, they do not represent complete generalization to unseen river locations or different environmental conditions. Further evaluation using geographically diverse river environments would provide stronger evidence regarding the model’s ability to generalize beyond the current dataset.

4.7 Robustness Under Reduced-Data Conditions

To test sensitivity to training-set size, YOLO26n was retrained with 75%, 50%, and 25% of the available training images; the validation and test data were kept fixed.

Table 4.7: Reduced-data robustness of the 50-epoch YOLO26n configuration.

Training Data	P	R	mAP@0.5	mAP@0.5:0.95
100%	0.859	0.797	0.866	0.556
75%	0.818	0.777	0.844	0.532
50%	0.690	0.707	0.742	0.460
25%	0.567	0.667	0.632	0.379

Table 4.7 and Fig. 4.4 show a progressive decline in performance as the training set was reduced. mAP@0.5 decreased only slightly from 0.866 to 0.844 when 75% of the training data was used, but dropped to 0.742 and 0.632 with 50% and 25% of the training data, respectively. The findings show that while a little decrease in training data had little impact on the model’s performance, a large decrease in data resulted in a noticeable decrease in detection capabilities. This experiment shows that detection reliability is directly affected by training data availability and that maintaining the robust performance of the model requires adequate data diversity. It is reasonable to assume that models trained on more varied datasets will perform well—or generalize effectively—beyond controlled laboratory situations, given the notable differences in item appearance, lighting conditions, and background environments observed in river settings.

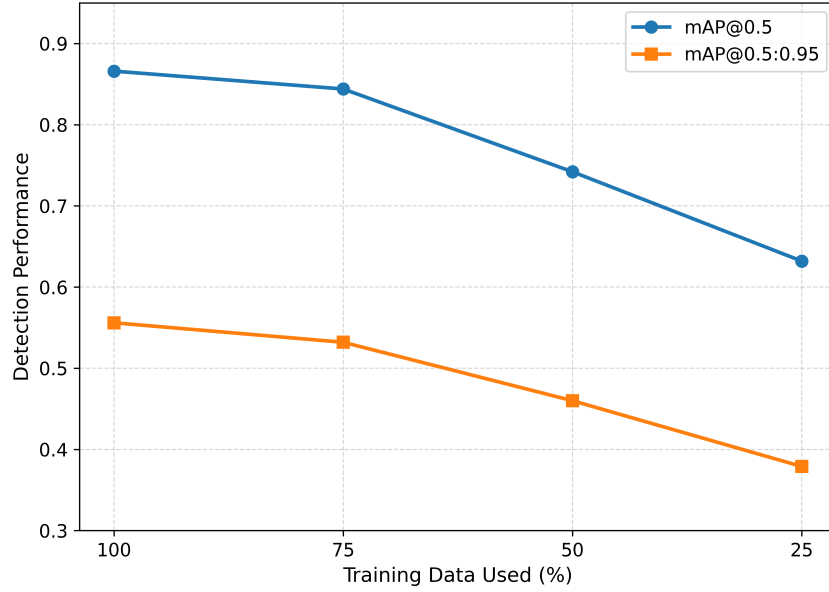


Figure 4.4: Detection performance trend under reduced-data conditions.

4.8 Final YOLO26n Model Performance

YOLO26n was chosen for longer training after the preceding benchmarks showed competitive accuracy at relatively low model complexity. Its training schedule was therefore extended from 50 to 100 epochs.

Table 4.8: Performance of YOLO26n under 50- and 100-epoch training configurations.

Model	P	R	mAP50	mAP50-95
YOLO26n (50 epochs)	0.859	0.797	0.866	0.556
YOLO26n (100 epochs)	0.919	0.844	0.915	0.608

Extending training to 100 epochs improved all four reported metrics (Table 4.8). Precision increased from 0.859 to 0.919, Recall from 0.797 to 0.844, mAP@0.5 from 0.866 to 0.915, and mAP@0.5:0.95 from 0.556 to 0.608. Under the current training configuration, the 100-epoch run achieved higher test-set performance than the 50-epoch configuration, indicating that the longer training schedule was beneficial in this experiment. The training and validation curves in Fig. 4.5 show continued improvement while remaining broadly aligned, without severe late-epoch divergence.

Based on the improved detection performance and the lightweight characteristics required for embedded deployment, the YOLO26n-100 checkpoint was selected for subsequent reliability analysis, explainable AI evaluation, and prototype integration. The confusion matrix in Fig. 4.6 provides an error-oriented view of the model predictions, showing that most plastic instances were correctly detected, while the remaining errors were mainly associated with missed detections and background-related false responses.

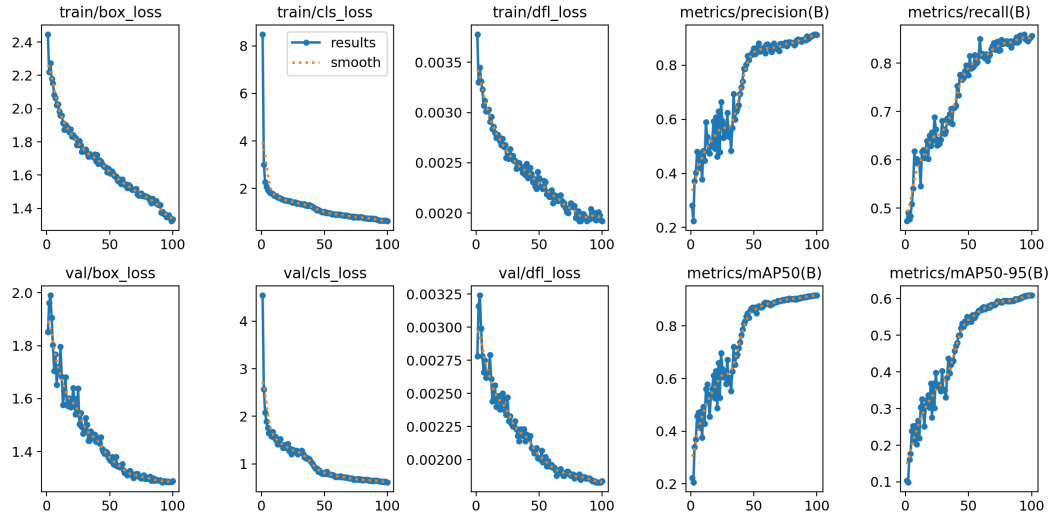


Figure 4.5: Training and validation behavior of the refined YOLO26n model during 100-epoch optimization.

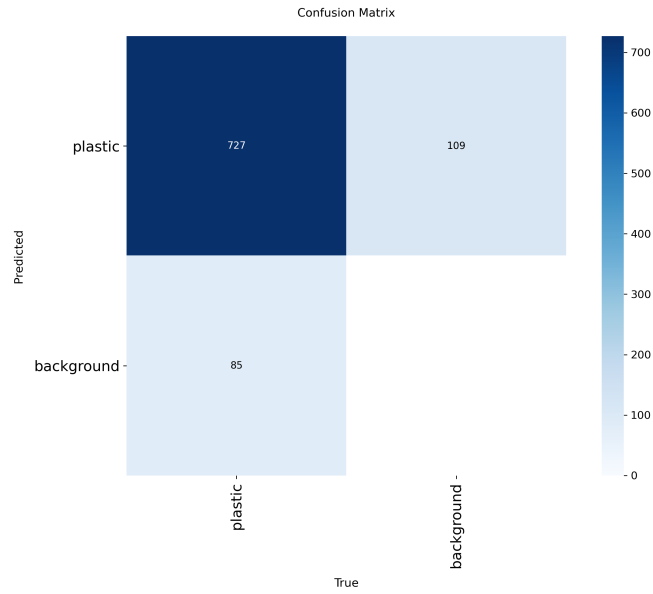


Figure 4.6: Confusion matrix of the 100-epoch YOLO26n model evaluated on the cleaned single-class test set.

4.9 Confidence Calibration Analysis

The held-out test predictions of YOLO26n-100 are summarized in Fig. 4.7. Most retained detections fell within the upper confidence range, with a clear concentration between 0.75 and 0.90.

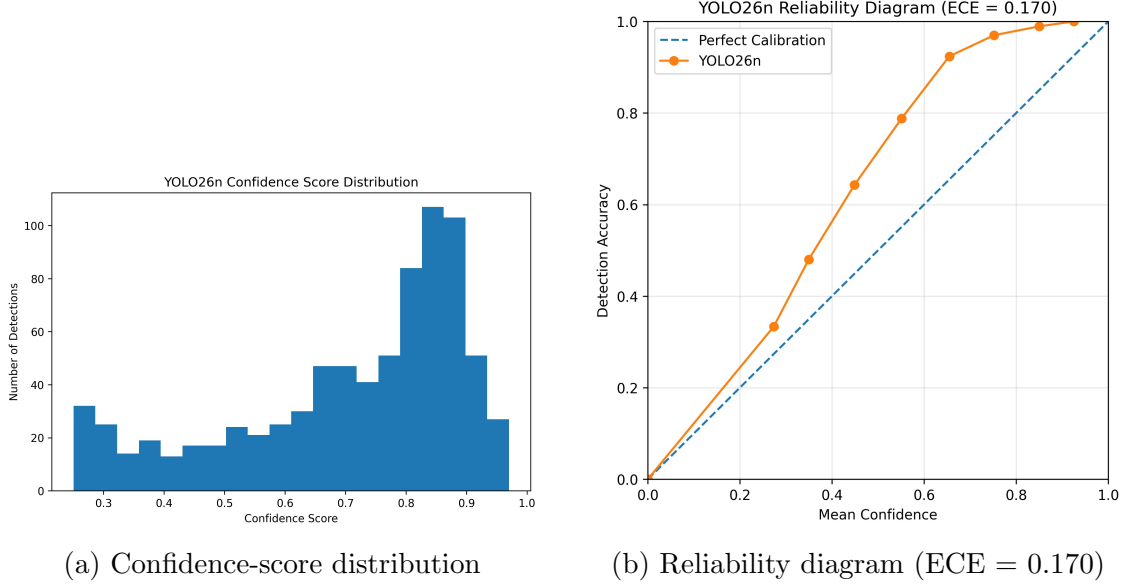


Figure 4.7: Confidence and calibration analysis of the selected YOLO26n-100 model.

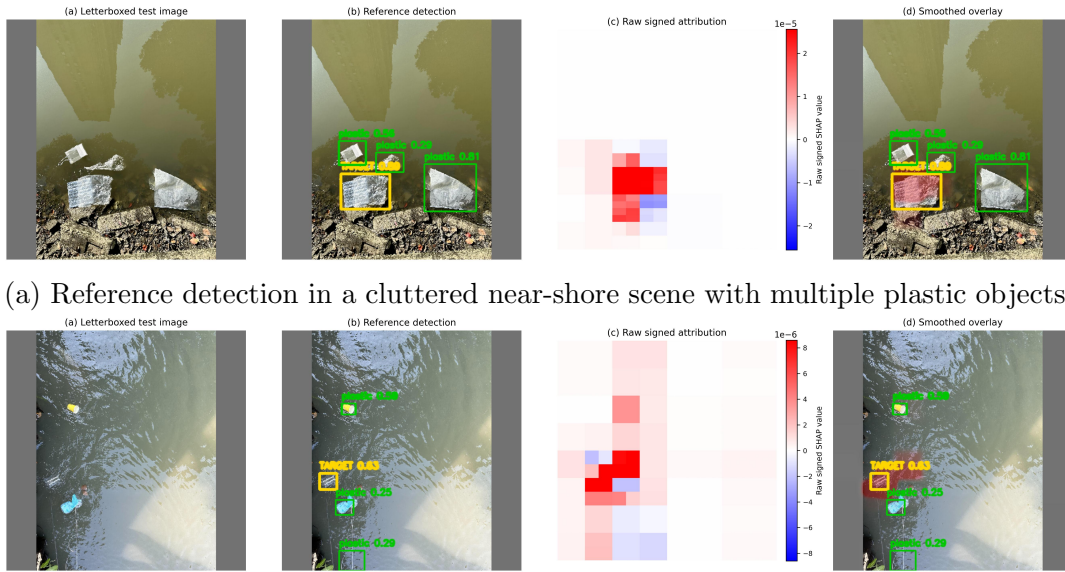
For the calibration analysis, the remaining 795 detections were filtered using a 0.25 confidence limit. The Expected Calibration Error (ECE) [35, 36] was used to summarize these predictions after they were divided into ten equal-width confidence bins and matched against ground truth at an IoU ≥ 0.50 . The resulting ECE was 0.1696; under-confidence under the chosen evaluation approach was shown by empirical precision continuously exceeding average confidence across all bins. The average confidence in the 0.80–0.90 confidence interval was 0.8500, while the empirical precision was 0.9893. Because predictions with confidence less than 0.25 and missing ground-truth objects were eliminated, it should be emphasized that this ECE number represents a threshold-conditioned, prediction-level calibration rather than a thorough, location-aware calibration assessment.

By analyzing the relationship between the expected confidence ratings of the model’s predictions and their actual accuracy, calibration analysis expands the evaluation process beyond the boundaries of standard classification accuracy. This evaluation is particularly important for autonomous systems because downstream actions, such as rover navigation and collection triggering, may depend on detection certainty. Poorly calibrated confidence values can lead to unnecessary movements or incorrect collection decisions; therefore, calibration analysis provides additional evidence regarding the reliability of the selected YOLO26n-100 model.

4.10 Explainable AI Analysis

SHAP-based attribution was evaluated on 21 test images using the YOLO26n-100 checkpoint [37]. For each image, the highest-confidence plastic detection above 0.25

was used as the reference target, with perturbed predictions matched to the same class at an IoU threshold of 0.30.



(a) Reference detection in a cluttered near-shore scene with multiple plastic objects.

(b) Small reference target under strong water reflection and multiple-object conditions.

Figure 4.8: Representative detection-specific SHAP examples generated using the YOLO26n-100 checkpoint. Each example shows the letterboxed test image, fixed reference detection, raw signed attribution map, and smoothed attribution overlay. Positive target-confidence contributions are shown in red, while negative contributions are shown in blue.

Fig. 4.8 shows two representative examples from the 21-image SHAP analysis. In both cases, attribution was distributed across the detected target and surrounding scene regions, consistent with the quantitative spatial-alignment results in Table 4.9.

Table 4.9 shows that masking the top 20% of positively attributed pixels reduced matched-target confidence by 0.688 ± 0.186 , with only small additional reductions at the 40% and 60% masking levels. Positive attribution was only partly concentrated within the target region, with $30.9\% \pm 23.7\%$ of the attribution mass inside the original box and $37.2\% \pm 26.2\%$ inside the expanded box. The pointing-game and centroid criteria were satisfied in 12 of 21 images. These results indicate that the analyzed detections relied on both target-localized and surrounding contextual information rather than only a single object region. However, the SHAP findings are limited to the 21 analyzed test images and the selected blur-based perturbation scheme; therefore, they should be interpreted as sample-level attribution evidence rather than a complete explanation of the detector’s general decision process.

The explainability analysis provides additional insight beyond conventional accuracy metrics by examining how the selected YOLO26n-100 model makes decisions in complex river environments. It is essential to determine if a model’s predictions largely depend on the target-specific region or on surrounding contextual features in order to confirm its dependability prior to real-world deployment. Particularly in difficult situations like reflections on water, complicated backgrounds, partial visibility, and visually similar environmental characteristics, this analysis aids in locating possible sources of failure.

Table 4.9: SHAP faithfulness and spatial-alignment results of YOLO26n-100 ($n = 21$).

Metric	Estimate	95% CI
<i>Faithfulness</i>		
Confidence drop (20%)	0.688 ± 0.186	[0.605, 0.762]
Confidence drop (40%)	0.725 ± 0.171	[0.650, 0.797]
Confidence drop (60%)	0.729 ± 0.152	[0.663, 0.793]
<i>Spatial alignment</i>		
Target-box attribution mass	0.309 ± 0.237	[0.211, 0.411]
Expanded-box attribution mass	0.372 ± 0.262	[0.260, 0.483]
Top-5% positive pixels	0.304 ± 0.343	[0.173, 0.470]
Pointing-game hit	0.571 (12/21)	[0.365, 0.755]
Centroid hit	0.571 (12/21)	[0.365, 0.755]

Note: Binary spatial metrics are displayed as proportions with Wilson 95% confidence intervals, whereas continuous metrics are expressed as mean $\pm$ population standard deviations with bootstrap 95% confidence intervals.

4.11 Qualitative Detection Analysis

Representative instances of single-class and multi-class detection were examined in addition to the quantitative evaluation results.

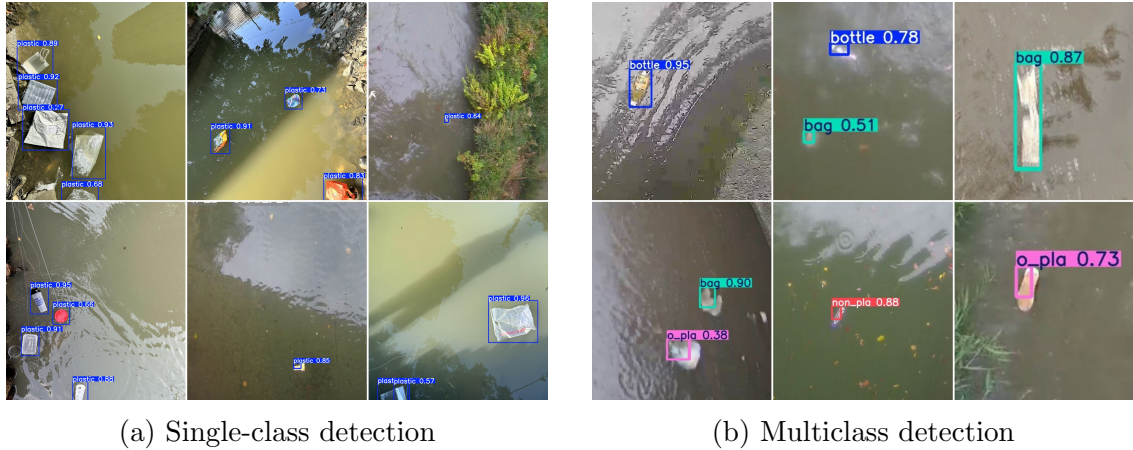


Figure 4.9: Representative qualitative detection results obtained under real riverine environmental conditions.

Successful identification over a range of illumination conditions, item sizes, water textures, and backdrop complexity is shown in Figure 4.9. However, detection remained challenging for reflective surfaces, partially submerged objects, and small distant targets. These selected examples illustrate model behavior and are not treated as independent evidence of robustness or generalization. However, qualitative analysis remains useful for understanding practical model behavior because certain failure patterns, such as reflection, occlusion, or small-object difficulty, may not be fully captured by aggregate numerical metrics.

4.12 Deployment-Oriented Evaluation

For deployment-oriented evaluation, YOLO26n-100 was compared with YOLO26s using model complexity, checkpoint size, CPU throughput, and detection accuracy.

Table 4.10: CPU workstation benchmark of the selected YOLO26 deployment configurations.

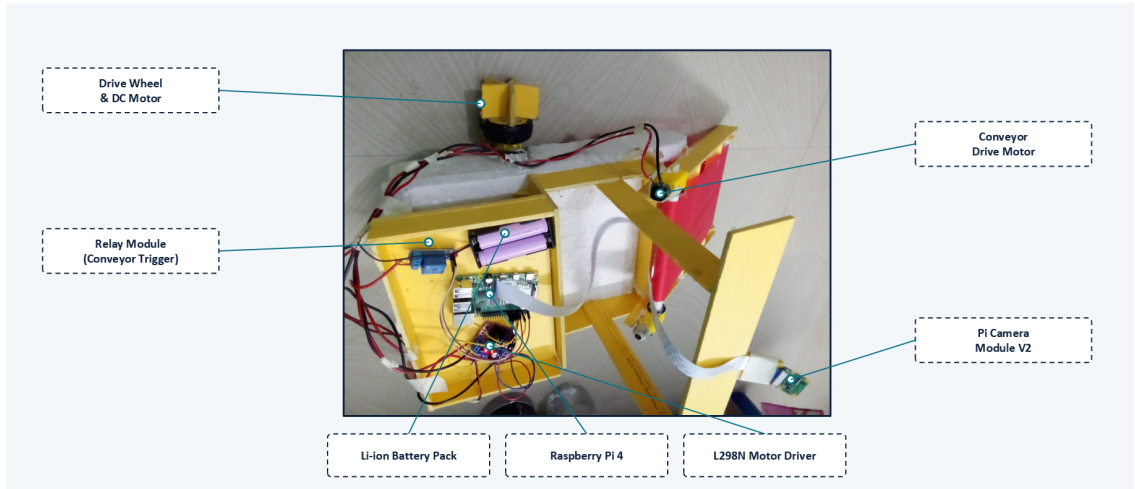
Model	Params (M)	GFLOPs	Size (MB)	FPS	mAP50	mAP50-95
YOLO26n-100	2.38	5.20	5.13	7.96	0.915	0.608
YOLO26s	9.47	20.50	19.36	3.30	0.876	0.580

YOLO26n-100 used 2.38M parameters, 5.20 GFLOPs, and a 5.13 MB checkpoint, whereas YOLO26s required 9.47M parameters, 20.50 GFLOPs, and 19.36 MB (Table 4.10). The smaller model also recorded 0.915 mAP@0.5, 0.608 mAP@0.5:0.95, and 7.96 FPS on the CPU benchmark. These results supported its selection as the deployment candidate. However, this comparison uses 100-epoch YOLO26n and 50-epoch YOLO26s checkpoints and should therefore be interpreted as a final-checkpoint comparison rather than an equal-training-budget experiment. Despite this limitation, the comparison provides useful practical information regarding the suitability of different model sizes for embedded deployment. The results support the decision to prioritize YOLO26n because the target application requires continuous inference on resource-constrained hardware.

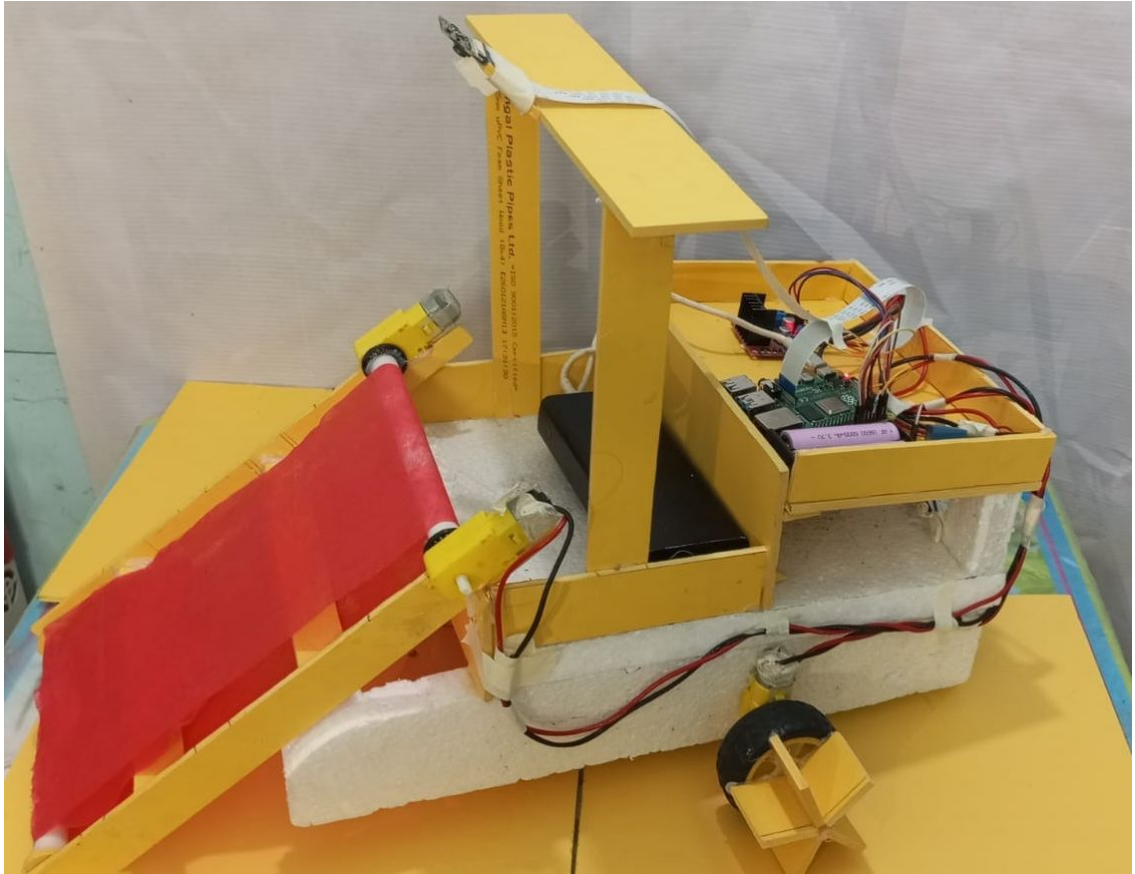
4.13 Prototype Integration and Proof-of-Concept Validation

The selected YOLO26n-100 model was integrated into the PLADICS prototype to demonstrate the detection-to-control workflow on a physical rover platform.

The selected YOLO26n-100 model was integrated into the PLADICS prototype to demonstrate the complete detection-to-control workflow on a physical rover platform. The prototype integrates AI-based perception, embedded computing, robotic control, collection actuation, and communication components within a single system architecture. This integration confirms the feasibility of connecting visual detection, decision-making, physical action, and monitoring into a unified framework rather than evaluating detection performance as an isolated software task. However, the current evaluation represents a proof-of-concept validation and does not indicate long-term autonomous field deployment. Although additional testing under different river environments, operating conditions, and extended mission durations is required, the prototype demonstrates the potential of extending deep learning-based detection toward practical environmental monitoring and plastic collection applications.



(a) Component layout and onboard hardware arrangement.



(b) Complete PLADICS prototype side view.

Figure 4.10: Developed PLADICS prototype integrating onboard computing, perception, control, and plastic collection mechanisms.

4.14 Summary

Across all experiments, the 100-epoch YOLO26n checkpoint provided the strongest overall test-set performance, achieving 0.919 Precision, 0.844 Recall, 0.915 mAP@0.5, and 0.608 mAP@0.5:0.95 while remaining among the lightest models evaluated. Cross-validation confirmed consistent internal performance with a mean mAP@0.5 of 0.860 ± 0.015 , whereas the reduced-data experiments demonstrated that moderate reductions in training data had limited impact but larger reductions resulted in clear performance degradation. Calibration analysis indicated mild systematic under-confidence with an ECE of 0.1696, and SHAP attribution analysis showed strong deletion sensitivity but only partial spatial concentration within detected regions, suggesting that the model relied on both target-localized and contextual information. Together with the deployment-oriented comparison and physical prototype validation, these findings supported the selection of YOLO26n-100 as the deployment model for the PLADICS rover. Overall, the experimental evaluation demonstrates that PLADICS provides a balanced approach between detection accuracy, computational efficiency, and practical system integration, highlighting the potential of lightweight AI-based robotic systems for future automated river-surface plastic monitoring and collection applications.

Chapter 5

Conclusion and Future Scope

5.1 Summary of the Work

In order to identify and gather floating plastic from river surfaces, this capstone project created an integrated framework called "PLADICS." In order to create a smooth operational workflow, the system combines computer vision-based perception, dataset refinement, model evaluation, embedded system deployment, robotic control, physical collection, and IoT-based monitoring. A thorough dataset preparation and refinement procedure was carried out in order to improve the reliability of model training and evaluation; lightweight YOLO versions, compact CNN-based detectors, and transformer-based detectors were then compared. 'YOLO26n' was chosen as the best lightweight model by striking a balance between detection performance and computing economy. To illustrate an efficient automated workflow, the selected model was optimized and implemented on a Raspberry Pi 4 using NCNN. It was then connected with ByteTrack-based tracking, rule-based rover navigation, and a conveyor-based collection system. In addition to traditional accuracy measures, other assessments were carried out to confirm the model's dependability, including robustness testing, SHAP-based explainability analysis, and confidence calibration. On the test dataset, the resulting "YOLO26n-100" model obtained precision of 0.919, recall of 0.844, mAP@0.5 of 0.915, and mAP@0.5:0.95 of 0.608. This illustrates the viability of incorporating IoT, robotics, embedded computers, and lightweight AI into an automated system for tracking and gathering plastic debris from river surfaces.

5.2 Limitations and Future Scope of This Research

Future study must address a number of the shortcomings of the established PLADICS framework, even though it shows that it is feasible to automatically detect and collect floating plastic from river surfaces. Plastic that is small, far away, partially submerged, or hidden by other items is still difficult for the existing detecting technology to detect. Environmental elements that can impact identification reliability include aquatic vegetation, illumination fluctuations, water surface reflections, and visually similar backgrounds. The results may not correctly reflect performance in unfamiliar rivers, under seasonal changes, or in diverse sensing situations because the current dataset solely reflects unique river settings. Currently, the rover's navigation system uses a rule-based approach that makes use of bounding boxes, confidence ratings, and object location data. Future studies should investigate learning-based navigation techniques to improve performance in more complicated

operational contexts, even if this method is clear and understandable.

Furthermore, rather than a long-term autonomous field deployment, the current prototype evaluation is a proof-of-concept implementation. The IoT monitoring system was verified in a local network environment, and the SHAP explainability study was carried out offline with a small number of test samples. By developing more extensive and varied river-specific datasets that cover a range of geographic locations, seasons, weather conditions, and camera views, future research could overcome these constraints. High-resolution training, multi-scale feature learning, and hard-negative sampling could enhance the detection of small and visually difficult plastic objects. Using learning-based navigation techniques, adaptive control strategies, and sophisticated object re-identification could improve tracking and navigation modules even more. Dependability in challenging environmental circumstances may be improved by additional sensing technologies including depth sensors, infrared cameras, and multi-modal sensing systems. The existing prototype may be developed into a useful, autonomous river-cleaning system with additional study into large-scale field testing, multi-rover deployment, cloud-based IoT monitoring, extended autonomous missions, and adaptive environmental learning.

Bibliography

- [1] M. MacLeod, H. P. H. Arp, M. B. Tekman, and A. Jahnke, “The global threat from plastic pollution,” *Science*, vol. 373, no. 6550, pp. 61–65, 2021.
- [2] United Nations Environment Programme, “Turning off the tap: How the world can end plastic pollution and create a circular economy,” technical report, United Nations Environment Programme, Nairobi, Kenya, 2023.
- [3] L. J. J. Meijer, T. van Emmerik, R. van der Ent, C. Schmidt, and L. Lebreton, “More than 1000 rivers account for 80% of global riverine plastic emissions into the ocean,” *Science Advances*, vol. 7, no. 18, p. eaaz5803, 2021.
- [4] T. H. M. van Emmerik, S. Kirschke, L. J. Schreyers, S. Nath, C. Schmidt, and K. Wendt-Potthoff, “Estimating plastic pollution in rivers through harmonized monitoring strategies,” *Marine Pollution Bulletin*, vol. 196, p. 115503, 2023.
- [5] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, “Ssd: Single shot multibox detector,” in *Computer Vision – ECCV 2016* (B. Leibe, J. Matas, N. Sebe, and M. Welling, eds.), vol. 9905 of *Lecture Notes in Computer Science*, (Cham), pp. 21–37, Springer International Publishing, 2016.
- [6] P. Mittal, “A comprehensive survey of deep learning-based lightweight object detection models for edge devices,” *Artificial Intelligence Review*, vol. 57, p. 242, 2024.
- [7] L. Chen and J. Zhu, “Water surface garbage detection based on lightweight yolov5,” *Scientific Reports*, vol. 14, no. 6133, 2024.
- [8] Z. Fan, Z. Qin, W. Liu, M. Chen, and Z. Qiu, “Ss-yolov8: A lightweight algorithm for surface litter detection,” *Applied Sciences*, vol. 14, no. 9283, 2024.
- [9] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, “DETRs beat YOLOs on real-time object detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 16965–16974, 2024.
- [10] B. Sun, H. Tang, L. Gao, K. Bi, and J. Wen, “RTDETR-MARD: A multi-scale adaptive real-time framework for floating waste detection in aquatic environments,” *Journal of Marine Science and Engineering*, vol. 13, no. 5, p. 996, 2025.
- [11] C. Zhang, J. Yue, J. Fu, and S. Wu, “River floating object detection with transformer model in real time,” *Scientific Reports*, vol. 15, p. 9026, 2025.

- [12] M. Nunkhaw and H. Miyamoto, “An image analysis of river-floating waste materials by using deep learning techniques,” *Water*, vol. 16, no. 10, p. 1373, 2024.
- [13] J. Cao *et al.*, “Edge-computing-driven internet of things: A survey,” *ACM Computing Surveys*, vol. 55, no. 8, pp. 1–36, 2023.
- [14] OASIS Open, “MQTT version 5.0,” oasis standard, Organization for the Advancement of Structured Information Standards, 2019.
- [15] T. Shehzadi *et al.*, “Object detection with transformers: A review,” *Sensors*, vol. 25, no. 19, p. 6025, 2025.
- [16] J. Wang and H. Zhao, “Improved YOLOv8 algorithm for water surface object detection,” *Sensors*, vol. 24, no. 15, p. 5059, 2024.
- [17] C. Peng, B. He, W. Xi, and G. Lin, “Improved YOLOv7 algorithm for floating waste detection based on GFPN and long-range attention mechanism,” *Wuhan University Journal of Natural Sciences*, vol. 29, no. 4, pp. 338–348, 2024.
- [18] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, “Ultralytics YOLO26: Unified real-time end-to-end vision models,” *arXiv preprint arXiv:2606.03748*, 2026.
- [19] G. Tikász *et al.*, “A lightweight real-time macroplastic detection framework based on yolov8 for river monitoring applications,” *Frontiers in Environmental Science*, 2025.
- [20] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “Bytetrack: Multi-object tracking by associating every detection box,” in *Computer Vision – ECCV 2022*, vol. 13682 of *Lecture Notes in Computer Science*, (Cham), pp. 1–21, Springer, 2022.
- [21] OpenJS Foundation and Node-RED Contributors, “Node-red: Low-code programming for event-driven applications,” 2026. Software project; accessed 2 August 2026.
- [22] B. Badams, U. U. Sheikh, S. A. R. Syed Abu Bakar, and N. Abdul Wahab, “FloYO-Net: Enhancing small floating waste detection in natural waters using atrous YOLOv5s,” *EMITTER International Journal of Engineering Technology*, vol. 13, no. 2, pp. 214–228, 2025.
- [23] Y. Cao, H. Luo, M. Wang, Y. Wang, H. Yan, *et al.*, “Enhanced YOLOv8 for accurate and efficient floating object detection on water surfaces,” *Scientific Reports*, vol. 16, p. 2907, 2026.
- [24] S. I. Sumon, M. E. H. Chowdhury, J.-U. K. Chowdhury, A. Ashraf, S. B. A. Kashem, M. E. Majid, M. Nashbat, A. Khandakar, M. Hasan-Zia, and A. K. A. Kunju, “Floating waste detection using deep learning: a comparative study of yolo, rt-detr, and faster r-cnn,” *Neural Computing and Applications*, vol. 38, p. 293, 2026.

- [25] X. Xu, C. Zhang, H. Huang, and L. Kong, “Improving performance for multi-category anthropogenic debris detection in river environments by using a size-tailored annotation approach and data augmentation,” *Scientific Reports*, vol. 16, p. 18400, 2026.
- [26] B. Jitnupong, J. Charoensuk, S. Bundasak, S. Somritjinda, N. Silabumrungrad, and J. Suraseing, “Detection and classification of floating waste on water surfaces using YOLO-based algorithms,” *Proceedings of Engineering and Technology Innovation*, vol. 33, pp. 106–119, 2026.
- [27] T. Zhou, J. Liu, N. Lian, Y. Chen, J. Liu, Z. Pei, J. Li, Z. Yin, J. Zhou, and Z. Li, “ESAK-YOLO: Multi-scale attention enhanced object detection for floating waste in complex water scenes,” *Scientific Reports*, 2026.
- [28] T. Kataoka, T. Yoshida, and N. Yamamoto, “Risid: River surface image dataset for instance segmentation of floating macroplastic debris,” *Data in Brief*, vol. 63, p. 112189, 2025.
- [29] C. Zhang, X. Xu, and Y. Xia, “Riverine plastic litter dataset,” 2024. Dataset.
- [30] Y. Gong, G. Liu, Y. Xue, R. Li, and L. Meng, “A survey on dataset quality in machine learning,” *Information and Software Technology*, vol. 162, p. 107268, 2023.
- [31] Y. Cheng, J. Zhu, M. Jiang, J. Fu, C. Pang, P. Wang, K. Sankaran, O. Onabola, Y. Liu, D. Liu, and Y. Bengio, “FloW: A dataset and benchmark for floating waste detection in inland waters,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 10953–10962, 2021.
- [32] Ultralytics, “Ultralytics yolo,” 2026. Software repository.
- [33] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, “YOLOv10: Real-time end-to-end object detection,” *arXiv preprint arXiv:2405.14458*, 2024.
- [34] R. Lyu, “Nanodet-plus: Super fast and lightweight anchor-free object detection model,” 2021. Software implementation.
- [35] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proceedings of the 34th International Conference on Machine Learning*, vol. 70 of *PMLR*, pp. 1321–1330, 2017.
- [36] F. Küppers, J. Kronenberger, A. Shantia, and A. Haselhoff, “Multivariate confidence calibration for object detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pp. 326–327, 2020.
- [37] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 4765–4774, 2017.